# Fondamenti di Programmazione

Spark

ottobre 2026 · volume 1

# Capire un programma, un passo alla volta

Un programma prende dei dati e li trasforma. Sembra facile dire se lo fa bene, finché non ci chiediamo che cosa significhi davvero «bene». Per rispondere dobbiamo dire con precisione che cosa vogliamo ottenere, seguire le operazioni una dopo l'altra e trovare un ragionamento che valga per tutti gli ingressi possibili, non soltanto per quelli che abbiamo provato. Questo libro costruisce un passaggio alla volta, sempre partendo da esempi piccoli, e richiede soltanto l'aritmetica e l'algebra della scuola.

## 1 Il percorso

Il viaggio comincia da una richiesta, da una procedura che la soddisfa e dai valori che cambiano mentre la eseguiamo. Sono la **specifica**, l'**algoritmo** e lo **stato**, i protagonisti del capitolo 1.

Per parlare di questi valori senza ambiguità ci serve un linguaggio preciso, e questo linguaggio è la **logica**. Prima impariamo a ragionare con frasi che possono essere vere o false (capitolo 2), poi a parlare di oggetti e a dire «tutti» ed «esiste» con i **quantificatori** (capitoli 3 e 4).

Con questi strumenti torniamo ai programmi. Studiamo che cosa fanno i comandi (capitolo 5) e colleghiamo il loro effetto alle proprietà che valgono prima e dopo l'esecuzione, grazie alle **triple di Hoare** (capitolo 6).

L'ultima parte è dedicata ai cicli, dove si concentrano le difficoltà più interessanti. Gli **invarianti** ci diranno che cosa rimane vero a ogni giro, le **funzioni di terminazione** perché il calcolo prima o poi finisce. Negli array useremo dei confini che si spostano per separare il lavoro già fatto da quello che resta da fare (capitoli 7–9).

## 2 Come leggere gli esempi

Non serve imparare subito ogni simbolo. Conviene leggere l'esempio, chiedersi che cosa vogliamo sapere e solo allora tornare alla definizione. Davanti a un assegnamento, fermati un momento e domandati quale valore viene letto e quale viene sostituito.

Nelle dimostrazioni distingui ciò che sappiamo prima di un passo da ciò che dobbiamo sapere dopo. Quando scegliamo un comando o un'uguaglianza, il testo prova sempre a spiegare perché proprio quella, e vale la pena cercare il motivo anche da soli.

Ogni esercizio ha una soluzione spiegata nell'ultima parte del libro. Prima di leggerla, prova almeno a scegliere il primo passo. Le sezioni «Per proseguire» sviluppano varianti secondarie e si possono lasciare per una seconda lettura.

## 3 Convenzioni e fonti

I naturali comprendono lo zero. Usiamo il piccolo linguaggio imperativo del corso con interi matematici, che non hanno limiti di grandezza. La divisione intera e le condizioni perché un'espressione abbia valore verranno precisate prima di usarle.

Il comando $x := E$ cambia il valore di $x$, mentre l'uguaglianza $x = E$ afferma una proprietà. Le *variabili di specifica* ricordano i valori iniziali e non cambiano durante una prova.

Il volume raccoglie le lezioni del corso fino al 31 ottobre 2026. Le spiegazioni aggiunte non sono citazioni del docente, e le fonti e le scelte editoriali sono registrate nei documenti che accompagnano il libro.

# Indice

**Capire un programma, un passo alla volta** . . . 2

**1 Algoritmi, specifiche e stati** . . . 5

1.1 Dal problema alla specifica . . . 5
1.2 Stato e assegnamento: che cosa cambia . . . 6
1.3 Algoritmi diversi, costi diversi . . . 7
1.4 Modelli di calcolo e limiti . . . 8

**2 Calcolo proposizionale: verità e conseguenza** . . . 10

2.1 Formule, connettivi e interpretazioni . . . 10
2.2 Modelli, tautologie e controesempi . . . 11
2.3 Equivalenze e dimostrazioni motivate . . . 12
2.4 Implicazione e polarità delle occorrenze . . . 13
2.5 Per proseguire: forme normali, risoluzione e NAND . . . 14

**3 Logica del primo ordine: individui e formule** . . . 16

3.1 Termini, predicati e quantificatori . . . 16
3.2 Portata, variabili libere e sostituzione . . . 17
3.3 Interpretazioni: i simboli ricevono un significato . . . 18
3.4 Formalizzare: tutti, alcuni, esattamente uno . . . 18

**4 Semantica e dimostrazioni con i quantificatori** . . . 20

4.1 Assegnamenti e valore di termini e formule . . . 20
4.2 Negazione, distribuzione e domini ristretti . . . 21
4.3 Regole di prova: individui particolari e generici . . . 22
4.4 Uguaglianza e asserzioni sugli stati . . . 24
4.5 Per proseguire: leggi sui domini e indebolimento . . . 24

**5 Stati, espressioni e comandi** . . . 26

5.1 Lo stato e la valutazione delle espressioni . . . 26
5.2 Assegnamento, sequenza e perdita di informazione . . . 27
5.3 Scelte e ripetizioni: if e while . . . 28
5.4 La semantica dei comandi e le esecuzioni senza risultato . . . 29

**6 Triple di Hoare e invarianti dei cicli** . . . 30

6.1 Precondizione, postcondizione e correttezza . . . 30
6.2 Conseguenza e definitezza delle espressioni . . . 31
6.3 Assegnamento: ragionare a partire dal risultato . . . 32
6.4 Sequenza: trovare l'asserzione intermedia . . . 32
6.5 Condizionale: utilizzare l'informazione della guardia . . . 33
6.6 Invariante e correttezza parziale del while . . . 33

**7 Correttezza totale e funzioni di terminazione** . . . 35

7.1 Una quantità naturale che diminuisce . . . 35
7.2 La regola del while in correttezza totale . . . 36
7.3 Divisione intera: completare la prova . . . 36
7.4 Somma dei primi interi: scegliere l'invariante . . . 37
7.5 Programmi annotati e proprietà non modificate . . . 38
7.6 Per proseguire: eliminare gli incroci . . . 39

**8 Costruire un ciclo: il significato delle variabili** . . . . . 40

8.1 Dalla postcondizione a una proprietà da mantenere . . . . . 40
8.2 Fattoriale: ricordare il prodotto già calcolato . . . . . 40
8.3 MCD: trasformare i dati senza cambiare la risposta . . . . . 41
8.4 Radice cubica: costruire potenze con differenze . . . . . 42
8.5 Fibonacci: tenere il valore che manca . . . . . 44
8.6 Per proseguire: seguire una ricorrenza che dimezza l'indice . . . . . 45

**9 Array: indici, aggiornamenti e zone** . . . . . 46

9.1 Array come funzioni sugli indici . . . . . 46
9.2 Aggiornamento selettivo: prima valutare, poi modificare . . . . . 47
9.3 Incrementare un array: un confine tra fatto e da fare . . . . . 47
9.4 Somma di un array: sostituire la fine con un confine mobile . . . . . 48
9.5 Bandiera a due colori: classificare senza perdere elementi . . . . . 49
9.6 Per proseguire: bandiera a tre colori . . . . . 50

**Appendice A Soluzioni ragionate** . . . . . 52

A.1 Capitolo 1 — Specifiche e stati . . . . . 52
A.2 Capitolo 2 — Calcolo proposizionale . . . . . 52
A.3 Capitolo 3 — Individui e formule . . . . . 52
A.4 Capitolo 4 — Semantica e quantificatori . . . . . 53
A.5 Capitolo 5 — Stati, espressioni e comandi . . . . . 53
A.6 Capitolo 6 — Triple di Hoare e invarianti . . . . . 54
A.7 Capitolo 7 — Correttezza totale . . . . . 54
A.8 Capitolo 8 — Costruire un ciclo . . . . . 54
A.9 Capitolo 9 — Array . . . . . 55

Capitolo 1

# Algoritmi, specifiche e stati

**In breve**
Prima di scrivere un programma bisogna sapere che cosa deve fare, e non è sempre così ovvio come sembra. In questo capitolo prendiamo una moltiplicazione fatta con sole somme e la usiamo per separare tre cose che di solito si confondono: il risultato che vogliamo, le istruzioni che lo producono e i valori che cambiano durante il calcolo. Alla fine guarderemo più da lontano, chiedendoci che cosa significhi calcolare e se esistano problemi che nessun programma può risolvere.

Supponiamo di dover calcolare $5 \cdot 3$ avendo a disposizione soltanto l'addizione. Possiamo sommare 5 per tre volte e ottenere 15. Il risultato è lo stesso di sempre, ma la strada per arrivarci dipende dalle operazioni che chi esegue il calcolo sa compiere.

Ci sono quindi due domande diverse, che conviene tenere separate fin dall'inizio. La prima riguarda il risultato che vogliamo, la seconda i passi che possono produrlo.

## 1.1 Dal problema alla specifica

«Trova il numero di telefono di una persona» sembra una richiesta chiarissima. Appena proviamo a eseguirla, però, nascono le domande. Che cosa facciamo se due persone hanno lo stesso nome? E se il nome non compare nell'elenco? Prima di scegliere i passi del calcolo dobbiamo sciogliere queste ambiguità, perché un esecutore automatico non può indovinare che cosa intendevamo.

Una **specifica** serve a questo. Dice quali dati sono ammessi all'inizio e quale relazione deve legare quei dati al risultato. Per il nostro prodotto la specifica è semplice: dati due interi non negativi $a$ e $b$, ottenere $a\,b$. Ammettiamo anche lo zero, che controlleremo esplicitamente. Notiamo che la specifica non dice nulla sugli strumenti da usare, né somme né moltiplicazioni, perché descrive soltanto il traguardo.

Per arrivare a una buona specifica aiuta farsi alcune domande. Quali dati servono davvero? Quali parti del problema possiamo affrontare separatamente? Abbiamo già risolto qualcosa di simile? Sono tre aspetti di ciò che si chiama **pensiero computazionale**, cioè l'*astrazione*, la *scomposizione* e il riconoscimento di *schemi*.

Fissato il traguardo, possiamo cercare una procedura così precisa da poter essere eseguita senza bisogno di interpretazioni. L'informatica studia come progettare e analizzare queste procedure, chiamate algoritmi, e quali problemi si possano risolvere con esse.

**Concetto chiave · Algoritmo e programma**
Un algoritmo descrive un procedimento mediante regole finite, non ambigue ed eseguibili da un esecutore adatto. Un programma lo esprime in un linguaggio di programmazione. Per un problema che richiede un risultato, chiediamo anche che l'esecuzione termini su tutti i dati ammessi e produca quel risultato.

La parola «finite» merita un momento di attenzione. Una regola come «finché rimangono somme da fare, eseguine una» è una sola regola, anche se durante l'esecuzione la usiamo molte volte. Che la descrizione sia finita e che l'esecuzione termini sono quindi due questioni distinte. Una ricetta che dice «aggiungi sale quanto basta», invece, non è un algoritmo, perché lascia indeterminato quando fermarsi, e un esecutore automatico ha bisogno di una condizione precisa.

Il lavoro procede poi per tappe. Dopo la *specifica* progettiamo l'algoritmo, lo traduciamo in un linguaggio, cioè lo *codifichiamo*, e infine lo eseguiamo. Se qualcosa non va, dobbiamo capire in quale tappa è nato l'errore, che può stare nella richiesta, nel procedimento, nella traduzione o nell'esecuzione. Questa ricerca si chiama **debugging**, e un programma detto debugger aiuta a osservare l'esecuzione un passo alla volta.

Anche i test hanno un limite che conviene conoscere subito. Un test riuscito ci dice che il programma funziona sul caso provato, e nient'altro. Per essere sicuri di tutti i casi ammessi, che spesso sono infiniti, servirà una dimostrazione.

## 1.2 Stato e assegnamento: che cosa cambia

Torniamo al prodotto per somme e chiediamoci che cosa dobbiamo ricordare durante il calcolo. Ci servono tre informazioni: il numero da aggiungere, quante aggiunte mancano e quanto abbiamo accumulato finora. Chiamiamole $A$, $B$ e $C$. Uno **stato** raccoglie i loro valori in un certo momento, e all'inizio è

$$\sigma = \{A \mapsto 5, B \mapsto 3, C \mapsto 0\}.$$

La scrittura $B \mapsto 3$ associa il nome $B$ al valore 3. Il nome resta lo stesso per tutto il calcolo e ci permette di ritrovare l'informazione, mentre il valore associato può cambiare.

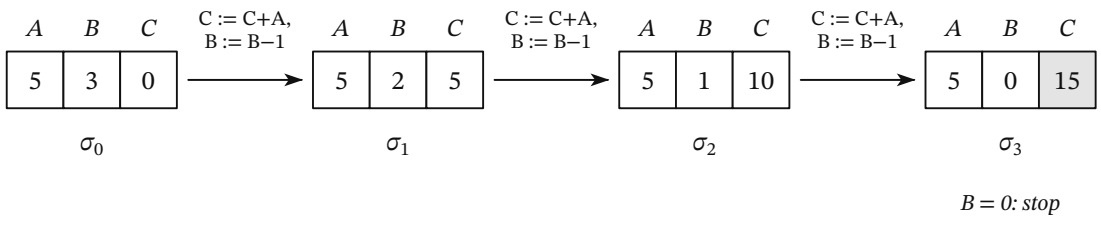
Ogni nome presente nello stato deve avere un solo valore, altrimenti non sapremmo che cosa leggere. Lo stato $\{x \mapsto 3, y \mapsto 3\}$ va benissimo, perché nomi diversi possono avere lo stesso valore. Lo stato $\{x \mapsto 3, x \mapsto 4\}$ invece non ha senso, perché alla domanda «quanto vale $x$?» darebbe due risposte. Non tutti i nomi possibili devono però comparire, e per questo si dice che uno stato è una **funzione parziale** dai nomi ai valori.

Lo strumento che cambia lo stato è l'**assegnamento**. Il comando $C := C + A$ legge i valori attuali di $C$ e di $A$, ne calcola la somma e la mette al posto del vecchio valore di $C$. Nello stato iniziale ottiene $C = 5$, mentre $A$ e $B$ restano 5 e 3. Non è un'equazione da risolvere, anche se le somiglia: le due $C$ parlano di due momenti diversi, prima e dopo il comando.

Possiamo ora scrivere tutto il procedimento.

```
C := 0;
while B > 0 do
  C := C + A;
  B := B - 1
endw
```

Il ciclo `while` controlla la sua *guardia* $B > 0$ prima di ogni giro. Se la guardia è vera esegue nell'ordine le due istruzioni del corpo e torna al controllo, se è falsa esce senza eseguirle. Così, se $B$ vale 0 fin dall'inizio, il corpo non viene mai eseguito e il risultato resta 0, che è il prodotto cercato.




Stati ai controlli della guardia per $A = 5, B = 3$. Tra due colonne si eseguono prima l'aggiunta e poi il decremento.

Ogni scelta del programma ha una ragione. Diminuiamo $B$ perché ci serve sapere quante aggiunte mancano, e partiamo con $C = 0$ perché prima della prima aggiunta non abbiamo accumulato nulla. Seguendo la figura, dopo il primo giro abbiamo fatto una somma e ne mancano due, dopo il secondo abbiamo 10 e ne manca una, dopo il terzo abbiamo 15 e non ne manca nessuna.

### 1.2.1 Ricordare il prodotto iniziale

Per controllare il risultato dobbiamo ricordarci da dove siamo partiti, anche se il programma cambia le sue variabili. Chiamiamo quindi $a$ il valore iniziale di $A$ e $b$ quello di $B$. Sono **valori di specifica**: non compaiono nel programma e rimangono fissi per tutto il ragionamento.

Con $a = 5$ e $b = 3$, dopo il primo giro abbiamo già prodotto 5 e ci restano due aggiunte da 5. Se sommiamo la parte calcolata e quella ancora da calcolare ritroviamo il prodotto iniziale,

$$5 + 5 \cdot 2 = 15.$$

Questa osservazione non dipende dai numeri scelti. A ogni controllo della guardia vale

$$C + A\,B = a\,b.$$

All'inizio è vera, perché $C = 0$, $A = a$ e $B = b$. Durante un giro aggiungiamo $A$ a $C$ e togliamo 1 a $B$, cioè la parte calcolata cresce di $A$ esattamente quando la parte mancante cala di $A$. Il conto lo conferma:

$$(C + A) + A(B - 1) = C + A\,B.$$

Quando il ciclo si ferma abbiamo $B = 0$, e l'uguaglianza diventa $C = a\,b$, il risultato che volevamo. Il ciclo si ferma davvero, perché $B$ parte da un naturale e scende di 1 a ogni giro finché arriva a zero.

Una proprietà che rimane vera a ogni giro in questo modo si chiama **invariante**. La incontreremo di nuovo nei capitoli 6 e 7, dove impareremo a dimostrarla con metodo e a usarla nelle prove dei cicli.

C'è un dettaglio che vale la pena notare. Scrivere come richiesta finale «$C = A\,B$» sarebbe sbagliato, perché alla fine $B = 0$ e quel prodotto non ricorda più nulla dell'ingresso. La richiesta corretta è $C = a\,b$, che parla dei valori iniziali. Se poi volessimo anche lasciare intatti gli ingressi, dovremmo dirlo nella specifica e far scendere un contatore ausiliario al posto di $B$.

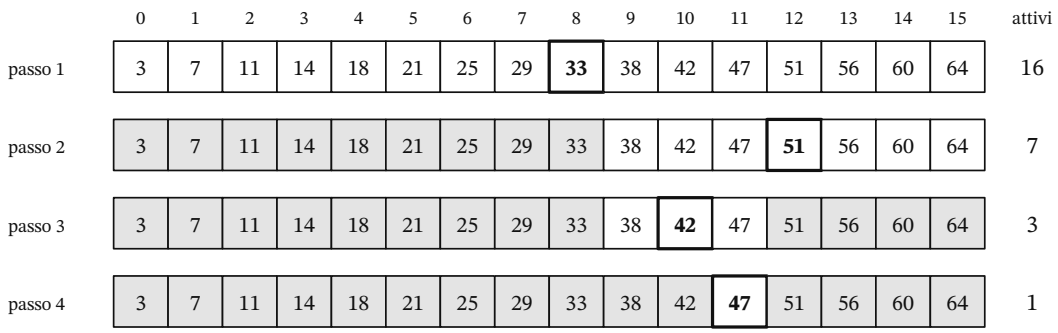
## 1.3 Algoritmi diversi, costi diversi

Il prodotto per somme richiede $b$ giri, e con numeri grandi sono molti. Possiamo fare meglio sfruttando una regolarità dei numeri. Se il primo fattore è pari, dimezzarlo e raddoppiare il secondo non cambia il prodotto. Se è dispari e maggiore di 1, togliamo un'unità e mettiamo da parte una copia del secondo fattore. Con $13 \cdot 11$ otteniamo

$$13 \cdot 11 = 12 \cdot 11 + 11 = 6 \cdot 22 + 11 = 3 \cdot 44 + 11.$$

Continuando allo stesso modo, $3 \cdot 44 = 2 \cdot 44 + 44 = 1 \cdot 88 + 44$, e il risultato è $88 + 44 + 11 = 143$. È la moltiplicazione egizia richiamata nella lezione introduttiva, un procedimento antichissimo che mostra bene come una proprietà dei numeri possa diventare un algoritmo più rapido.

Un secondo esempio viene dalla ricerca in un elenco ordinato. Invece di scorrere i nomi uno per uno, confrontiamo quello cercato con il nome centrale. Se il cercato viene prima, possiamo scartare tutti i nomi successivi, e se viene dopo scartiamo tutti i precedenti. È l'ordinamento a renderci liberi di scartare metà elenco, e senza di esso questo passo non sarebbe lecito.




Ricerca di 47: i valori confrontati sono 33, 51, 42, 47. La zona attiva si restringe perché i valori sono ordinati.

Se contiamo come un passo ogni confronto con l'elemento centrale, a ogni passo i candidati si riducono circa della metà, mentre la ricerca sequenziale li esamina uno alla volta. Entrambi i metodi trovano il nome giusto, ma uno impiega molti meno passi. Dire che un algoritmo è corretto, dunque, non dice ancora quanto costa eseguirlo, perché **correttezza** ed **efficienza** sono proprietà diverse.

## 1.4 Modelli di calcolo e limiti

Finora abbiamo parlato di «esecutore» in modo intuitivo. Possiamo rendere precisa questa idea senza costruire un calcolatore fisico. Una **macchina di Turing** ha un nastro di simboli, una testina e un insieme finito di regole, e a ogni passo legge un simbolo, ne scrive uno, si sposta e cambia il proprio stato di controllo.

Fra le macchine di Turing ce n'è una speciale, la **macchina universale**. Riceve la descrizione di un'altra macchina insieme al suo ingresso e ne simula il comportamento. È l'idea che sta dietro ai nostri calcolatori, dove uno stesso esecutore può svolgere programmi diversissimi.

### 1.4.1 Modi diversi di descrivere il calcolo

Il **λ-calcolo** di Church descrive il calcolo in un modo che non somiglia affatto a un nastro, usando soltanto funzioni e applicazioni di funzioni. Eppure i due modelli calcolano esattamente le stesse funzioni, e la loro equivalenza è un teorema.

La **tesi di Church–Turing** fa un passo ulteriore e afferma che questi modelli catturano tutti i procedimenti che intuitivamente chiameremmo calcoli eseguibili. Si chiama tesi e non teorema perché la nozione intuitiva di calcolo non ha una definizione matematica indipendente con cui confrontarla.

L'architettura di **von Neumann**, su cui si basano i calcolatori di oggi, tiene in memoria sia i dati sia le istruzioni. Quando ne discutiamo la capacità teorica di calcolo immaginiamo una memoria estendibile a piacere, anche se un calcolatore reale ha naturalmente risorse finite. Il corso cita anche altri modelli, come i sistemi di Post e le funzioni ricorsive, e accenna ai modelli quantistici e biologici, che qui ci limitiamo a nominare.

### 1.4.2 Una richiesta che nessun algoritmo risolve in generale

Viene spontaneo pensare che ogni richiesta precisa si possa risolvere con un algoritmo, ma non è così. Il **problema dell'arresto** chiede di decidere, per ogni programma e ogni ingresso, se l'esecuzione terminerà. Si dimostra che è indecidibile: non esiste alcun algoritmo che termini sempre e dia sempre la risposta corretta su tutti questi casi. Questo non ci impedisce di dimostrare che un programma particolare termina, ed è esattamente ciò che faremo nei capitoli 7 e 8.

### 1.4.3 Come si organizza un programma

Lo stesso calcolo si può organizzare in modi molto diversi, chiamati **paradigmi di programmazione**. La tabella raccoglie quelli citati nel corso.

| Paradigma | Idea principale |
|---|---|
| Imperativo, come C | Comandi che cambiano lo stato |
| Funzionale | Funzioni e applicazioni |
| A oggetti | Oggetti che raccolgono dati e operazioni |
| Logico, come Prolog | Relazioni e inferenze |
| Concorrente | Processi che interagiscono |

Nel resto del libro ci occuperemo del paradigma imperativo, quello dei comandi che cambiano lo stato. Prima di studiarne gli altri comandi, però, ci serve un modo preciso per descrivere le proprietà degli stati, e questo è il compito della logica.

---

**Verifica 1**

**1.** Nell'algoritmo del prodotto, parti da $A = 4, B = 2$. Scrivi gli stati ai controlli della guardia, spiega il ruolo di $C$ e indica perché $C = A\,B$ non è la richiesta finale corretta.

**2.** Vuoi ottenere il maggiore fra due interi senza modificare gli ingressi. Scrivi una specifica che distingua ingressi, risultato e vincoli. Spiega perché la richiesta non stabilisce ancora l'algoritmo.

# Calcolo proposizionale: verità e conseguenza

**In breve**
Ogni giorno traiamo conclusioni da frasi come «se piove, porto l'ombrello», e quasi sempre lo facciamo bene. A volte però un ragionamento che sembra filare contiene un salto nascosto. Il calcolo proposizionale ci dà un modo sicuro per accorgercene, e allo stesso tempo il linguaggio con cui scriveremo le condizioni dei programmi.

«Se Corrado va al cinema, ci va anche Antonio. Se Antonio va al cinema, ci va anche Bruno.» Possiamo concludere che, se ci va Corrado, ci va Bruno? Sì, e il motivo è semplice da seguire: quando Corrado va, la prima frase ci porta ad Antonio e la seconda da Antonio a Bruno.

Non possiamo invece concludere che, se va Bruno, vada anche Corrado, perché Bruno potrebbe benissimo andarci da solo. Il calcolo proposizionale trasforma questa intuizione in un controllo preciso. Una conclusione segue dalle premesse quando è impossibile che le premesse siano tutte vere e la conclusione falsa.

## 2.1 Formule, connettivi e interpretazioni

Una **proposizione** è una frase dichiarativa a cui attribuiamo un valore, vero oppure falso. Una domanda o un ordine non sono proposizioni, perché non ha senso chiedersi se siano veri. Una frase come «$x+1=2$» è un caso intermedio: diventa vera o falsa soltanto quando sappiamo quanto vale $x$, e ce ne occuperemo con la logica del primo ordine.

Per ora indichiamo le proposizioni semplici con le lettere $p, q, r, \ldots$ e le combiniamo con i *connettivi*. La formula $\neg P$ si legge «non $P$», $P \wedge Q$ si legge «$P$ e $Q$» e $P \vee Q$ significa «almeno una fra $P$ e $Q$». Questa disgiunzione è inclusiva, cioè ammette che le due parti siano vere insieme. Infine $P \Rightarrow Q$ si legge «se $P$, allora $Q$» e $P \equiv Q$ si legge «$P$ se e solo se $Q$».

Il «se» merita una sosta, perché nel linguaggio comune è un po' sfuggente. Chi dice «se piove, porto l'ombrello» esclude un solo caso, quello in cui piove e lui non ha l'ombrello. Nei giorni senza pioggia non ha promesso nulla, e può portarlo o lasciarlo a casa senza smentirsi. Per questo $P \Rightarrow Q$ è falsa soltanto quando $P$ è vera e $Q$ falsa. L'equivalenza $P \equiv Q$ è invece vera quando i due valori coincidono.

Indichiamo i due valori con t e f. La tabella seguente definisce tutte le operazioni.

| $P$ | $Q$ | $\neg P$ | $P \wedge Q$ | $P \vee Q$ | $P \Rightarrow Q$ | $P \equiv Q$ |
|---|---|---|---|---|---|---|
| t | t | f | t | t | t | t |
| t | f | f | f | t | f | f |
| f | t | t | f | t | t | f |
| f | f | t | f | f | t | t |

### 2.1.1 Come si costruisce una formula

Non ogni sequenza di simboli è una formula, e conviene dire con precisione quali lo sono. Si parte dalle lettere e dalle due costanti $T$ e $F$, che indicano la formula sempre vera e quella sempre falsa e, a differenza delle lettere, non hanno un valore da scegliere. Se abbiamo già due formule possiamo collegarle con $\wedge$, $\vee$, $\Rightarrow$ oppure $\equiv$, e se ne abbiamo una possiamo negarla.

Le stesse regole si applicano poi ai risultati, e ancora ai risultati dei risultati, ed è per questo che la costruzione si dice **ricorsiva**. L'insieme di queste regole è la **sintassi** del linguaggio, che si può scrivere come una grammatica libera dal contesto. Così $p \wedge \vee$ non è una formula, perché dopo $\wedge$ manca la seconda formula da collegare.

Le parentesi mostrano in che ordine è stata costruita la formula. Per risparmiarne qualcuna, nel corso si stabilisce che $\neg$ lega più forte di $\wedge$ e $\vee$, che a loro volta legano più forte di $\Rightarrow$, che lega più forte di $\equiv$. Fra $\wedge$ e $\vee$ non si sceglie una precedenza, e lo stesso vale per due implicazioni consecutive, quindi in questi casi le parentesi vanno scritte.

Non sono un dettaglio di forma. Se $P$ e $Q$ sono false, $(P \Rightarrow \neg P) \Rightarrow Q$ è falsa, mentre $P \Rightarrow (\neg P \Rightarrow Q)$ è vera, perché spostando le parentesi abbiamo cambiato la domanda logica.

### 2.1.2 Dare un valore alle formule

Un'**interpretazione** $I$ assegna a ogni lettera un valore booleano, e da lì possiamo calcolare il valore di qualunque formula seguendone la costruzione. Prendiamo $I(p) = \mathrm{t}$ e $I(q) = I(r) = \mathrm{f}$, e valutiamo $(p \wedge q) \vee \neg r$. Prima calcoliamo le due parti, $p \wedge q$, che è falsa, e $\neg r$, che è vera, e poi la loro disgiunzione, che è vera. Per il valore di una formula $P$ nell'interpretazione $I$ scriviamo $[\![P]\!]_I$.

Una **tavola di verità** ripete questo calcolo per tutte le interpretazioni possibili. Con $n$ lettere distinte le interpretazioni sono $2^n$, perché ogni lettera ha due valori possibili indipendentemente dalle altre.

## 2.2 Modelli, tautologie e controesempi

Un'interpretazione che rende vera una formula $P$ si chiama **modello** di $P$, e scriviamo $I \vDash P$. A seconda di quanti modelli hanno, le formule si dividono in tre famiglie.

Una formula è **soddisfacibile** se ha almeno un modello. Per esempio $p \Rightarrow q$ è vera quando $p$ e $q$ sono entrambe vere, e questo basta. Una **tautologia** è vera in ogni interpretazione, come $p \vee \neg p$, perché qualunque valore abbia $p$ una delle due parti è vera. La formula $p \Rightarrow q$ non è una tautologia, perché diventa falsa con $p$ vera e $q$ falsa. Una **contraddizione**, infine, non ha nessun modello, come $p \wedge \neg p$, dato che una proposizione e la sua negazione non possono essere vere insieme.

Con queste parole possiamo dire con precisione che cosa significa che una conclusione segue dalle premesse.

**Concetto chiave · Conseguenza logica**
$\Gamma \vDash Q$ significa che ogni interpretazione che rende vere tutte le premesse di $\Gamma$ rende vera anche $Q$.
Per confutare la conseguenza basta un **controesempio**: premesse tutte vere e conclusione falsa.

Torniamo agli amici al cinema, con le premesse $c \Rightarrow a$ e $a \Rightarrow b$. Per vedere che $c \Rightarrow b$ segue, proviamo a costruire un controesempio e guardiamo dove si blocca. La conclusione sarebbe falsa soltanto con $c$ vera e $b$ falsa. La prima premessa, per essere vera, ci obbliga allora ad avere $a$ vera, ma con $a$ vera e $b$ falsa la seconda premessa diventa falsa. Il controesempio non esiste, e quindi la conclusione segue.

Con $b \Rightarrow c$ il tentativo invece riesce. Basta prendere $b$ vera e $a$, $c$ false: le due premesse sono vere e la conclusione è falsa.

Le stesse premesse non ci permettono nemmeno di concludere che nessuno vada al cinema, perché l'interpretazione in cui ci vanno tutti e tre le rende vere. In italiano la frase «Antonio va *solo se* va Bruno» si scrive $a \Rightarrow b$, e dice che la presenza di Bruno è necessaria perché ci vada Antonio, non che basti. Una condizione necessaria non è automaticamente sufficiente.

Quando le premesse sono in numero finito, $P_1, \ldots, P_n$, c'è un modo comodo di riassumere tutto questo. La conseguenza vale esattamente quando la formula $(P_1 \wedge \cdots \wedge P_n) \Rightarrow Q$ è una tautologia, perché questa formula è falsa proprio nei controesempi. Potremmo quindi controllare ogni ragionamento con una tavola di verità, ma il numero delle righe raddoppia a ogni lettera nuova. Le leggi logiche ci offrono una strada più agile.

## 2.3 Equivalenze e dimostrazioni motivate

Una **legge** è una tautologia che usiamo per giustificare i passaggi di un ragionamento. Le più utili sono equivalenze, e il motivo è il *principio di sostituzione*: se $P \equiv Q$ è una legge, possiamo sostituire $P$ con $Q$ dentro una formula più grande senza cambiarne il valore in nessuna interpretazione. Per esempio possiamo sempre scrivere $\neg P \vee Q$ al posto di $P \Rightarrow Q$, perché le due formule sono false nello stesso unico caso, con $P$ vera e $Q$ falsa.

### 2.3.1 Leggi per trasformare una formula

Alcune leggi esprimono fatti che usiamo senza pensarci. Scambiare l'ordine dei termini di una congiunzione o di una disgiunzione non cambia nulla, e questa è la **commutatività**. Quando ripetiamo lo stesso connettivo possiamo anche spostare le parentesi, grazie all'**associatività**. E ripetere due volte la stessa condizione non aggiunge richieste, come dice l'**idempotenza**:

$$P \wedge P \equiv P, \qquad P \vee P \equiv P.$$

Le costanti permettono altre semplificazioni. Aggiungere a una congiunzione una condizione già vera non cambia nulla, e lo stesso vale se aggiungiamo a una disgiunzione una possibilità sempre falsa. Sono le leggi di **unità**,

$$P \wedge T \equiv P, \qquad P \vee F \equiv P.$$

Al contrario, una congiunzione con una parte falsa fallisce tutta, e una disgiunzione con una parte vera è vera tutta. Per questo $F$ e $T$ si dicono **elementi assorbenti** delle due operazioni:

$$P \wedge F \equiv F, \qquad P \vee T \equiv T.$$

La **distributività** permette di separare i due casi di una disgiunzione,

$$P \wedge (Q \vee R) \equiv (P \wedge Q) \vee (P \wedge R),$$

e vale anche la legge che si ottiene scambiando i ruoli di $\wedge$ e $\vee$.

Le leggi che riguardano la negazione sono forse le più usate. La doppia negazione dice che $\neg\neg P \equiv P$, e le leggi di De Morgan dicono

$$\neg(P \wedge Q) \equiv \neg P \vee \neg Q, \qquad \neg(P \vee Q) \equiv \neg P \wedge \neg Q.$$

Si capiscono bene chiedendosi quando una frase fallisce. «Entrambi» fallisce appena fallisce almeno uno dei due, mentre «almeno uno» fallisce soltanto se falliscono entrambi. Allo stesso modo un'implicazione fallisce soltanto quando la premessa è vera e la conclusione falsa, cioè $\neg(P \Rightarrow Q) \equiv P \wedge \neg Q$. Infine $P \equiv Q$ equivale sia a $(P \Rightarrow Q) \wedge (Q \Rightarrow P)$ sia a $(P \wedge Q) \vee (\neg P \wedge \neg Q)$.

### 2.3.2 Usare le leggi con una meta

Distribuire a caso raramente aiuta, ma a volte è esattamente la mossa giusta. Vogliamo semplificare $P \vee (\neg P \wedge Q)$. La lettera $P$ compare due volte, una volta da sola e una negata, e sappiamo che $P \vee \neg P$ è sempre vera. Conviene allora distribuire per avvicinare le due occorrenze:

$$P \vee (\neg P \wedge Q) \equiv (P \vee \neg P) \wedge (P \vee Q) \equiv T \wedge (P \vee Q) \equiv P \vee Q.$$

Il primo passo usa la distributività, il secondo il *terzo escluso* $P \vee \neg P \equiv T$ e l'ultimo la legge di unità. Abbiamo ottenuto una **legge di complemento**. L'altra legge della stessa famiglia, $P \wedge (\neg P \vee Q) \equiv P \wedge Q$, si dimostra allo stesso modo ed è proposta nella verifica alla fine del capitolo.

Le leggi di **assorbimento** si capiscono quasi a colpo d'occhio:

$$P \wedge (P \vee Q) \equiv P, \qquad P \vee (P \wedge Q) \equiv P.$$

Nella prima, chiedere $P$ rende già vera $P \vee Q$, che quindi non aggiunge nulla. Nella seconda, il caso $P \wedge Q$ è già compreso nel caso $P$. Non serve imparare tutte le leggi a memoria, perché ogni volta si tratta di cercare quella che rende la formula più semplice.

### 2.3.3 Dalle premesse alla conclusione

Per dimostrare un'implicazione possiamo assumere la premessa e cercare di arrivare alla conclusione. La regola più semplice è il **modus ponens**: da $P \Rightarrow Q$ e $P$ segue $Q$, perché l'unico caso con $Q$ falsa renderebbe falsa una delle premesse.

Il **modus tollens** procede all'indietro: da $P \Rightarrow Q$ e $\neg Q$ segue $\neg P$. Il ragionamento è per assurdo, perché se $P$ fosse vera costringerebbe anche $Q$ a esserlo. Lo stesso fatto si esprime con la **controposizione**, $(P \Rightarrow Q) \equiv (\neg Q \Rightarrow \neg P)$, che non scambia semplicemente i due lati ma li nega entrambi.

Ci sono altri due schemi che useremo spesso. La prova per assurdo di $P \Rightarrow Q$ assume $P \wedge \neg Q$, l'unico caso che potrebbe falsificare l'implicazione, e mostra che porta a una contraddizione. La prova per casi sceglie invece una proposizione $R$ e dimostra la stessa conclusione sia supponendo $R$ sia supponendo $\neg R$, che insieme esauriscono tutte le possibilità.

## 2.4 Implicazione e polarità delle occorrenze

Sappiamo che $x > 10 \Rightarrow x > 0$. Dire «maggiore di 10» è più impegnativo che dire «maggiore di 0», perché lo soddisfano meno numeri, e per questo diciamo che è una condizione più *forte*. Se dentro la richiesta $x > 10 \wedge R$ la sostituiamo con quella più debole otteniamo una richiesta più facile da soddisfare, e infatti

$$(x > 10 \wedge R) \Rightarrow (x > 0 \wedge R).$$

Sotto una negazione succede il contrario. «Non maggiore di 0» implica «non maggiore di 10», mentre il viceversa fallisce, per esempio con $x = 5$:

$$\neg(x > 0) \Rightarrow \neg(x > 10).$$

La negazione ha invertito il verso dell'implicazione. Lo stesso accade a sinistra di una freccia. La richiesta $A \Rightarrow R$ diventa più impegnativa se indeboliamo $A$, perché dovremo garantire $R$ in un numero maggiore di casi.

### 2.4.1 Seguire un'occorrenza nella formula

Per capire in quale verso agisce una sostituzione, dobbiamo sapere in che posizione si trova la lettera che sostituiamo. Prendiamo come esempio

$$\Phi = \neg(P \Rightarrow Q) \wedge R.$$

Possiamo leggerne la costruzione come un albero. In cima c'è la congiunzione, a sinistra troviamo una negazione e sotto di essa un'implicazione, mentre a destra troviamo direttamente $R$. La tabella riassume la strada che porta a ciascuna lettera.

| Occorrenza | Percorso dalla congiunzione | Inversioni |
|---|---|---|
| $P$ | $\wedge, \neg,$ antecedente di $\Rightarrow$ | 2 |
| $Q$ | $\wedge, \neg,$ conseguente di $\Rightarrow$ | 1 |
| $R$ | $\wedge$ | 0 |

Per arrivare a $P$ attraversiamo prima la negazione e poi il lato sinistro dell'implicazione. Le inversioni sono due, e due inversioni si annullano, quindi diciamo che questa occorrenza è **positiva**. Per arrivare a $Q$ attraversiamo la stessa negazione ma poi il lato destro dell'implicazione, che non inverte. Resta una sola inversione e l'occorrenza è **negativa**. Per $R$ non incontriamo inversioni, ed è positiva.

Possiamo controllare il risultato riscrivendo $\Phi$ come $(P \wedge \neg Q) \wedge R$, dove i tre ruoli si vedono a occhio nudo.

> **CONCETTO CHIAVE · Polarità di un'occorrenza**
> Partiamo dal segno positivo e seguiamo il percorso fino all'occorrenza scelta. Ogni negazione e ogni passaggio nell'antecedente di $\Rightarrow$ cambia il segno. $\wedge$, $\vee$ e il conseguente lo lasciano invariato. Prima eliminiamo gli eventuali $\equiv$.

Il segno dipende dal posto che una lettera occupa, non dal suo nome, quindi due copie di $P$ nella stessa formula possono avere segni diversi.

### 2.4.2 Usare il segno per sostituire

Ora possiamo dire che cosa succede quando sostituiamo. Supponiamo che valga $A \Rightarrow B$. Se sostituiamo $A$ con $B$ in una posizione positiva, la formula iniziale implica quella nuova. In una posizione negativa vale il contrario, e la formula nuova implica quella iniziale.

Per esempio sappiamo che $Q \wedge R \Rightarrow Q$. Nel conseguente di $P \Rightarrow (Q \wedge R)$ non incontriamo inversioni, quindi la sostituzione va nel verso diretto:

$$(P \Rightarrow (Q \wedge R)) \Rightarrow (P \Rightarrow Q).$$

Sappiamo anche che $P \Rightarrow P \vee Q$. Se sostituiamo nell'antecedente di $P \Rightarrow R$ incontriamo un'inversione, e il verso si capovolge:

$$((P \vee Q) \Rightarrow R) \Rightarrow (P \Rightarrow R).$$

Il risultato ha senso anche a parole. La formula a sinistra garantisce $R$ sia quando vale $P$ sia quando vale $Q$, e quindi a maggior ragione lo garantisce nel solo caso $P$.

### 2.4.3 Tenere presenti le ipotesi

Quando abbiamo assunto $P$ possiamo trattarla come vera, sostituirla con $T$ e scrivere $P \wedge Q \equiv T \wedge Q \equiv Q$. Bisogna però ricordare che questo risultato dipende dall'ipotesi. Quello che abbiamo dimostrato è

$$P \Rightarrow ((P \wedge Q) \equiv Q),$$

e non l'equivalenza $(P \wedge Q) \equiv Q$ senza condizioni, che infatti è falsa quando $P$ è falsa e $Q$ vera.

## 2.5 Per proseguire: forme normali, risoluzione e NAND

Un **letterale** è una lettera oppure la sua negazione, come $p$ o $\neg q$. Una **forma normale congiuntiva** (FNC) è una congiunzione di disgiunzioni di letterali, per esempio

$$(p \vee \neg q) \wedge r.$$

Una **forma normale disgiuntiva** (FND) scambia i ruoli, ed è una disgiunzione di congiunzioni di letterali come $(p \wedge q) \vee \neg r$.

Ogni formula si può portare in queste forme. Si eliminano prima $\Rightarrow$ e $\equiv$, poi si spingono le negazioni fino alle lettere con le leggi di De Morgan e infine si distribuisce. Il procedimento può allungare molto la formula, e la forma che si ottiene non è unica.

### 2.5.1 Risolvere due clausole

La **risoluzione** parte da due clausole come $P \vee Q$ e $\neg P \vee R$ e ne ricava $Q \vee R$. Il motivo è una prova per casi. Se $P$ è falsa la prima clausola impone $Q$, e se è vera la seconda impone $R$, quindi in ogni caso vale almeno una fra $Q$ e $R$.

Con le tre clausole $P \vee Q$, $\neg P \vee R$ e $\neg Q \vee R$ possiamo fare due passi. Risolvendo le prime due su $P$ otteniamo $Q \vee R$, e risolvendo questa con la terza su $Q$ otteniamo $R$. La risoluzione è alla base di molti metodi automatici di dimostrazione e della programmazione logica.

### 2.5.2 Quanti connettivi sono necessari?

Abbiamo usato cinque connettivi, ma ne servono davvero così tanti? Un insieme di connettivi si dice **funzionalmente completo** se permette di esprimere ogni funzione booleana, cioè ogni possibile scelta dei risultati in una tavola di verità.

L'insieme $\{\neg, \wedge\}$ è completo. Le leggi di De Morgan ci danno $\vee$, e con una FND possiamo raccogliere esattamente le righe della tavola in cui vogliamo il valore vero. Per la funzione sempre falsa basta $P \wedge \neg P$. Per lo stesso motivo è completo $\{\neg, \vee\}$, e lo è anche $\{\neg, \Rightarrow\}$, perché $P \vee Q$ si può scrivere come $\neg P \Rightarrow Q$.

Sorprendentemente basta un solo connettivo, il **NAND**, definito da $P \uparrow Q \equiv \neg(P \wedge Q)$. Ricostruiamo prima la negazione e poi la congiunzione,

$$P \uparrow P \equiv \neg P, \qquad (P \uparrow Q) \uparrow (P \uparrow Q) \equiv P \wedge Q,$$

e così ritroviamo l'insieme completo $\{\neg, \wedge\}$. Nella pratica conviene comunque usare più connettivi, perché le formule restano molto più leggibili.

---

**Verifica 2**

**1.** Con $p$: «piove» ed $e$: «esco», traduci «esco solo se non piove». La formula ottenuta implica «se non piove, esco»? Cerca un caso che distingua le due richieste.

**2.** Semplifica $P \wedge (\neg P \vee Q)$ mostrando le leggi usate. Poi stabilisci il segno delle occorrenze di $P$, $Q$, $R$ in $(P \vee \neg Q) \Rightarrow \neg R$.

# Logica del primo ordine: individui e formule

**In breve**
Con le sole lettere proposizionali possiamo dire «piove» o «esco», ma non possiamo dire «ogni numero pari ha un successore dispari», perché non vediamo i numeri di cui stiamo parlando. La logica del primo ordine apre le proposizioni e mostra gli oggetti al loro interno. In questo capitolo impariamo a scriverne le formule, a capire a quale quantificatore appartiene ogni variabile e a tradurre con cura frasi con «tutti», «alcuni» ed «esattamente uno».

Consideriamo la frase «tutti i numeri pari hanno un successore dispari». Potremmo provare a introdurre una lettera per il caso di 0, una per il caso di 2, una per il caso di 4, e così via, ma non finiremmo mai e soprattutto non avremmo scritto la regola generale.

Quello che ci manca è la possibilità di distinguere gli ingredienti della frase, cioè il numero, la proprietà di essere pari e l'operazione che porta al successore. La **logica del primo ordine** aggiunge questi ingredienti, insieme a due nuove parole, «per ogni» ed «esiste».

## 3.1 Termini, predicati e quantificatori

Per prima cosa scegliamo gli oggetti di cui vogliamo parlare, che formano il **dominio**. Nel nostro esempio è l'insieme dei naturali $\mathbb{N} = \{0, 1, 2, \dots\}$. Ci servono poi la costante 1, la funzione somma, i predicati pari e dispari e una variabile $x$. Con questi ingredienti la frase diventa

$$\forall x.(\text{pari}(x) \Rightarrow \text{dispari}(x+1)),$$

che si legge così: qualunque numero scegliamo per $x$, se è pari allora il suo successore è dispari. Il simbolo $\forall$ è il **quantificatore universale**. Per dire invece che almeno un numero è pari usiamo il **quantificatore esistenziale** e scriviamo $\exists x.\text{pari}(x)$.

Nell'esempio compaiono quattro tipi di simboli diversi. Le **costanti** sono nomi per oggetti fissati, come 1. Le **variabili** sono posti per oggetti ancora da scegliere, come $x$. Le **funzioni** rappresentano operazioni, come la somma, e i **predicati** rappresentano proprietà o relazioni, come l'essere pari. Insieme ai connettivi e ai quantificatori formano il linguaggio del primo ordine.

Ogni funzione e ogni predicato richiede un numero preciso di argomenti, chiamato **arietà**. Il successore ne vuole uno, la somma due, e così anche il predicato «minore», che confronta due numeri. Un predicato con zero argomenti non ha bisogno di nessun oggetto e si comporta come una lettera proposizionale.

### 3.1.1 Termini e formule

La scrittura $s(s(x))$ indica il successore del successore di $x$. È un numero, non una frase, e quindi non ha senso chiedersi se sia vera o falsa. Le scritture che indicano oggetti si chiamano **termini**, e si costruiscono a partire dalle costanti e dalle variabili, che sono già termini. Se una funzione $f$ richiede $n$ argomenti e abbiamo già i termini $t_1, \dots, t_n$, possiamo formare il nuovo termine $f(t_1, \dots, t_n)$. Per arrivare a $s(s(x))$, per esempio, applichiamo $s$ a $x$ e poi di nuovo al risultato.

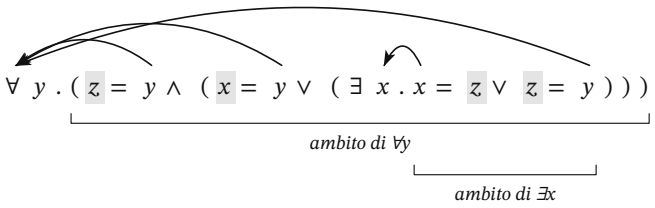
Le frasi vere e proprie nascono quando applichiamo un predicato a dei termini. Così otteniamo le **formule atomiche**, come $\text{pari}(s(s(x)))$ oppure $x < y$. Da queste, dalle costanti $T$ e $F$ e dai connettivi costruiamo formule più grandi esattamente come nel capitolo precedente. L'unica regola nuova riguarda i quantificatori: da una formula $P$ possiamo costruire $\forall x.P$ ed $\exists x.P$. Quando ci sono più quantificatori uguali di seguito li abbreviamo, scrivendo per esempio $\forall x, y.P$.

**Concetto chiave · Termine e formula**
Un termine indica un individuo; una formula esprime una condizione vera o falsa. Le funzioni prendono individui e restituiscono individui; i predicati prendono individui e restituiscono valori booleani. $x+1$ è un termine, $\text{pari}(x+1)$ una formula. $\text{pari}(x)+1$ non rispetta questi ruoli.

## 3.2 Portata, variabili libere e sostituzione

In una formula $\forall x.P$, la parte $P$ su cui agisce il quantificatore si chiama la sua **portata**. Un'occorrenza di $x$ è **legata** se sta nella portata di un quantificatore su $x$, altrimenti è **libera**. Quando ci sono più quantificatori annidati con lo stesso nome, un'occorrenza appartiene a quello più interno che la contiene.

Nella formula $(\forall x.p(x)) \wedge q(x,y)$ la prima $x$ è legata, mentre la seconda $x$ e la $y$ sono libere. La stessa lettera può quindi avere occorrenze con ruoli diversi. Una formula senza variabili libere si dice **chiusa**, altrimenti **aperta**, e la differenza è sostanziale. La formula aperta $x+1=2$ dipende dal valore che diamo a $x$ dall'esterno, mentre la formula chiusa $\exists x.x+1=2$ afferma che un valore adatto esiste, ed è vera o falsa una volta per tutte.




Le frecce identificano il quantificatore che lega ogni occorrenza. Nella formula illustrata tutte le $y$ sono legate, tutte le $z$ libere; la $x$ esterna a $\exists x$ rimane libera.

### 3.2.1 Sostituire senza catturare

Spesso vogliamo mettere un termine al posto di una variabile. La notazione $P[t/x]$ indica che sostituiamo il termine $t$ alle occorrenze *libere* di $x$ in $P$, lasciando stare quelle che appartengono a un quantificatore interno su $x$. C'è però una precauzione da prendere. Nessuna variabile libera del termine inserito deve finire nella portata di un quantificatore che la lega, perché altrimenti cambierebbe significato. Questo errore si chiama **cattura**.

Un esempio mostra perché la precauzione serve. Nella formula $P=\exists y.x<y$ vogliamo mettere $y$ al posto di $x$. Se lo facessimo ingenuamente otterremmo $\exists y.y<y$, dove la $y$ che volevamo lasciare libera è stata catturata dal quantificatore, e la formula dice ora una cosa completamente diversa. La soluzione è cambiare prima il nome della variabile legata, scrivendo $\exists z.x<z$, e solo dopo sostituire. Otteniamo $\exists z.y<z$, che sui naturali afferma correttamente che esiste un numero più grande di $y$.

Cambiare il nome di una variabile legata è sempre lecito, purché il nuovo nome non crei legami nuovi, e questa operazione si chiama **ridenominazione delle variabili legate**. In formule, se $z$ non compare in $P$,

$$\forall x.P \equiv \forall z.P[z/x].$$

La sostituzione tocca soltanto le occorrenze libere di $x$ in $P$, cioè proprio quelle che dipendono dal quantificatore esterno. Il nome della variabile legata è solo un segnaposto, e ciò che conta è quali occorrenze dipendono da quel quantificatore.

## 3.3 Interpretazioni: i simboli ricevono un significato

La formula $\forall x.p(x)$, da sola, non è né vera né falsa, perché non sappiamo di quali oggetti parliamo né che cosa significhi $p$. Per darle un valore dobbiamo scegliere un dominio non vuoto $D$ e dare un significato a ogni simbolo. Queste scelte formano un'**interpretazione**, che scriviamo $I = (D, \alpha)$.

Ogni costante indica un elemento di $D$. Una funzione con $n$ argomenti prende $n$ elementi del dominio e ne produce uno, e deve farlo per ogni scelta degli argomenti, cioè è una funzione *totale* $D^n \to D$. Qui $D^n$ è l'insieme delle liste ordinate di $n$ elementi di $D$. Un predicato con $n$ argomenti prende gli stessi dati ma risponde vero o falso, ed è quindi una funzione $D^n \to \text{Bool}$. L'uguaglianza ha sempre il significato usuale, e due termini sono uguali quando indicano lo stesso individuo.

Il dominio conta anche per gli oggetti che non hanno un nome. Se $D = \{5, 10, 15\}$ e $p$ significa «essere multiplo di 5», la formula $\forall x.p(x)$ è vera. Se allarghiamo il dominio a $\mathbb{N}$ e lasciamo a $p$ lo stesso significato, la formula diventa falsa, e 7 è un controesempio. Avere costanti soltanto per 5, 10 e 15 non toglie gli altri numeri dal dominio su cui variano i quantificatori.

Due osservazioni completano il quadro. Costanti diverse possono indicare lo stesso individuo, a meno che non imponiamo esplicitamente che siano distinte. E un simbolo dall'aspetto familiare come $+$ non ha automaticamente il significato aritmetico, che va dichiarato. Nel seguito, quando lavoreremo con numeri e programmi, useremo sempre il significato usuale.

## 3.4 Formalizzare: tutti, alcuni, esattamente uno

Tradurre una frase italiana in una formula richiede un po' di attenzione, e conviene vederlo su esempi concreti. Prendiamo come dominio le persone e due predicati, $S(x)$ per «$x$ segue FPL» e $C(x)$ per «$x$ va al cinema».

«Tutti gli studenti di FPL vanno al cinema» diventa $\forall x.(S(x) \Rightarrow C(x))$. Una persona che non segue FPL non smentisce la frase, perché la frase non parla di lei, ed è esattamente ciò che l'implicazione garantisce. «Qualche studente di FPL va al cinema» diventa invece $\exists x.(S(x) \wedge C(x))$, perché la persona che cerchiamo deve soddisfare entrambe le condizioni.

Questi due schemi sono così frequenti che hanno un'abbreviazione, la **quantificazione ristretta**:

$$(\forall x\colon S(x).C(x)) \equiv (\forall x.(S(x) \Rightarrow C(x))),$$

$$(\exists x\colon S(x).C(x)) \equiv (\exists x.(S(x) \wedge C(x))).$$

Scambiare i connettivi è un errore frequente, e vale la pena vedere che cosa produce. Con un'implicazione dentro l'esistenziale basterebbe trovare una persona che non segue FPL per rendere vera la formula, anche se nessuno studente andasse al cinema. Con una congiunzione dentro l'universale, al contrario, pretenderemmo che tutte le persone del mondo seguano FPL.

### 3.4.1 Dire che l'eccezione è una sola

La frase «tutti tranne uno» contiene due richieste, perché un'eccezione deve esistere e deve essere unica. Conviene dare prima un nome alla proprietà dell'eccezione, $E(x) = S(x) \wedge \neg C(x)$, cioè uno studente di FPL che non va al cinema. Allora «esattamente uno studente di FPL non va al cinema» si scrive

$$\exists x.(E(x) \wedge \forall y.(E(y) \Rightarrow y = x)).$$

La prima parte afferma che un'eccezione c'è, la seconda che ogni eccezione coincide con quella. In altre parole, tutti gli altri studenti vanno al cinema. Senza la seconda parte avremmo detto soltanto che le eccezioni sono almeno una.

### 3.4.2 Una relazione fra più individui

Il dominio e il vocabolario si scelgono in base alla domanda. Per dire che «due persone sono parenti se hanno un antenato comune» ci servono due predicati binari, $A(z, x)$ per «$z$ è antenato di $x$» e $R(x, y)$ per «$x$ e $y$ sono parenti». L'antenato comune non ha un nome, e diventa una variabile quantificata:

$$\forall x, y.((\exists z.(A(z, x) \land A(z, y))) \Rightarrow R(x, y)).$$

La frase dice «se», e l'implicazione va in una sola direzione. Con «se e solo se» dovremmo scrivere anche il verso opposto. Formalizzare significa quindi prima capire il senso della frase, e poi rileggere la formula per controllare che dica esattamente quello.

---

**Verifica 3**

**1.** Nella formula $(\forall x.p(x, y)) \land (\exists y.q(x, y))$ individua le occorrenze libere e legate. Sostituisci poi $y$ alle occorrenze libere di $x$, evitando catture.

**2.** Sui naturali formalizza «ogni numero pari ha il successore del successore pari», usando la funzione $s$ di successore. Formalizza poi «esattamente un elemento soddisfa $p$», spiegando il compito dell'uguaglianza.

# Semantica e dimostrazioni con i quantificatori

**In breve**
Dire che una proprietà vale «per ogni» numero è facile, ma come si controlla una frase che parla di infiniti casi? La risposta passa da un'idea semplice, quella di dare per un momento un valore a una variabile e vedere che cosa succede. Da lì ricaveremo le leggi per negare e spostare i quantificatori, e le regole con cui si dimostra una proprietà generale o si usa un oggetto di cui conosciamo soltanto l'esistenza.

Per sapere se $x + 1 = 4$ è vera dobbiamo sapere quale numero rappresenta $x$. Per sapere se $\exists x.x + 1 = 4$ è vera, invece, dobbiamo chiederci se qualche numero rende vera la condizione, e quindi provare dei valori. In entrambi i casi l'ingrediente è lo stesso, cioè dare temporaneamente un valore a una variabile, e su questa idea costruiremo la valutazione di tutte le formule.

## 4.1 Assegnamenti e valore di termini e formule

Fissiamo un'interpretazione $I = (D, \alpha)$. Un **assegnamento** $\rho$ associa a ogni variabile un elemento del dominio $D$. Per provare un valore diverso per una sola variabile scriviamo $\rho[d/x]$, che indica l'assegnamento uguale a $\rho$ su tutte le altre variabili ma che dà a $x$ il valore $d$. Per esempio, se $\rho(x) = 2$ e $\rho(y) = 10$, l'assegnamento $\rho[3/x]$ dà 3 a $x$ e lascia 10 a $y$.

Il valore di un termine, che indichiamo con $\alpha_\rho(t)$, si calcola seguendone la costruzione. Una variabile vale $\rho(x)$ e una costante vale $\alpha(c)$. Per un termine $f(t_1, \ldots, t_n)$ calcoliamo prima i valori dei termini interni e poi applichiamo ai risultati la funzione $\alpha(f)$. Se $s$ è il successore sui naturali e $\rho(x) = 2$, il termine $s(s(x))$ vale 4, perché partiamo da 2, passiamo a 3 e arriviamo a 4.

Per una formula atomica valutiamo i termini e applichiamo il predicato. Se $p$ significa «essere pari», la formula $p(s(s(x)))$ è vera con questo assegnamento, perché 4 è pari. I connettivi seguono le stesse tabelle del capitolo 2, e per il valore di una formula scriviamo $I_\rho(P)$. Resta da dire che cosa fanno i quantificatori, ed è qui che l'idea di provare dei valori diventa una definizione.

**Concetto chiave · Semantica dei quantificatori**
$I_\rho(\forall x.P)$ è vero se $I_{\rho[d/x]}(P)$ è vero per *ogni* $d \in D$. $I_\rho(\exists x.P)$ è vero se ciò accade per *almeno un* $d \in D$. Altrimenti il rispettivo valore è falso.

Su un dominio finito come $\{a, b, c\}$ la definizione è molto concreta. Un universale si comporta come la congiunzione dei tre casi, mentre un esistenziale si comporta come la loro disgiunzione. Un elemento che rende vera la portata di un esistenziale si chiama **testimone**, e uno che rende falsa la portata di un universale si chiama controesempio.

### 4.1.1 Un esempio con due quantificatori

Prendiamo $D = \{a, b\}$, con $Q$ vera su entrambi gli elementi e $P(x, y)$ vera soltanto quando $x = a$. Vogliamo sapere se è vera

$$\exists x.(Q(x) \wedge \forall y.P(x, y)).$$

Ci serve un elemento che soddisfi entrambe le richieste, e proviamo con $a$. La condizione $Q(a)$ è vera. Per l'universale interno dobbiamo controllare tutti i valori di $y$, cioè $P(a,a)$ e $P(a,b)$, e sono vere entrambe. Quindi $a$ è un testimone e la formula è vera. Con $b$ il controllo sarebbe fallito, ma questo non toglie nulla al testimone che abbiamo già trovato.

| $P(x,y)$ | $y=a$ | $y=b$ |
|---|---|---|
| $x=a$ | t | t |
| $x=b$ | f | f |

La tabella mostra bene il ruolo dei due quantificatori. All'universale interno serve un'intera riga vera, mentre all'esistenziale esterno basta trovarne una.

### 4.1.2 Formule valide

Una formula dipende soltanto dai valori assegnati alle sue variabili libere. Una formula chiusa, quindi, non dipende affatto da $\rho$, e possiamo scrivere semplicemente $I(P)$. Le nozioni di modello, soddisfacibilità e conseguenza logica si estendono come nel capitolo 2.

Una formula chiusa è **valida** se è vera in tutte le interpretazioni. Per una formula aperta la richiesta è ancora più forte, perché deve essere vera in ogni interpretazione e per ogni assegnamento alle variabili libere, ed è in questo senso che useremo le leggi schematiche. Essere vera sui naturali con le operazioni usuali è molto meno esigente. La formula $\forall x.x+1>x$, per esempio, è vera sui naturali, ma non lo è per qualunque significato si dia ai simboli.

## 4.2 Negazione, distribuzione e domini ristretti

Negare «tutti soddisfano $P$» significa dire che almeno uno non la soddisfa, e negare «almeno uno soddisfa $P$» significa dire che tutti la falliscono. Sono le leggi di De Morgan per i quantificatori:

$$\neg(\forall x.P) \equiv \exists x.\neg P, \qquad \neg(\exists x.P) \equiv \forall x.\neg P.$$

Con la quantificazione ristretta il filtro rimane al suo posto, e scriviamo $\neg(\forall x\colon R.P) \equiv \exists x\colon R.\neg P$. Non neghiamo $R$, perché il controesempio va cercato proprio fra gli elementi ammessi.

Un caso limite merita attenzione. Se nessun elemento soddisfa $R$, la formula $\forall x\colon R.P$ è **vacuamente vera**, perché non esiste nessun elemento ammesso che possa smentirla. Per lo stesso motivo $\exists x\colon R.P$ è falsa, dato che manca qualsiasi testimone. Così la frase «tutti gli elementi dell'array vuoto sono positivi» è vera, e non afferma affatto che nell'array ci sia qualche elemento.

Un universale si distribuisce sulla congiunzione, perché ogni individuo soddisfa entrambe le condizioni esattamente quando tutti soddisfano la prima e tutti la seconda. In modo simmetrico l'esistenziale si distribuisce sulla disgiunzione:

$$(\forall x.(P \wedge Q)) \equiv ((\forall x.P) \wedge (\forall x.Q)),$$

$$(\exists x.(P \vee Q)) \equiv ((\exists x.P) \vee (\exists x.Q)).$$

### 4.2.1 Perché alcuni versi non si possono invertire

Con gli scambi opposti, cioè l'esistenziale sulla congiunzione e l'universale sulla disgiunzione, vale soltanto un verso. Un testimone che soddisfa entrambe le proprietà si può usare separatamente per ciascuna, e quindi

$$(\exists x.(P \wedge Q)) \Rightarrow ((\exists x.P) \wedge (\exists x.Q)).$$

Il viceversa invece non vale. Sui naturali esistono numeri pari ed esistono numeri dispari, ma nessun numero è le due cose insieme, perché i due esistenziali hanno testimoni diversi.

Per l'universale la situazione è simile. Se tutti soddisfano $P$, oppure tutti soddisfano $Q$, allora ciascuno soddisfa almeno una delle due:

$$((\forall x.P) \vee (\forall x.Q)) \Rightarrow \forall x.(P \vee Q).$$

Anche qui il viceversa fallisce. Ogni intero è pari o dispari, ma questo non significa che tutti gli interi siano pari, né che siano tutti dispari.

### 4.2.2 Una parte che non dipende dalla variabile

Se $x$ non è libera in $Q$, il valore di $Q$ non cambia mentre $x$ varia, e possiamo portare questa parte fuori dal quantificatore:

$$(\forall x.(P \wedge Q)) \equiv ((\forall x.P) \wedge Q),$$

$$(\exists x.(P \vee Q)) \equiv ((\exists x.P) \vee Q).$$

Queste leggi usano un'ipotesi che finora era rimasta sullo sfondo, cioè che il dominio non sia vuoto. La stessa ipotesi permette di togliere un quantificatore da una formula che non dipende affatto da $x$, scrivendo $\forall x.Q \equiv Q$ ed $\exists x.Q \equiv Q$. Se ammettessimo domini vuoti, $\forall x.F$ sarebbe vera ed $\exists x.T$ falsa, e non potremmo più sostituirle con $F$ e con $T$.

### 4.2.3 L'ordine cambia la scelta del testimone

Due quantificatori dello stesso tipo si possono scambiare liberamente, quindi $\forall x.\forall y.P \equiv \forall y.\forall x.P$, e lo stesso vale per due esistenziali. Con quantificatori di tipo diverso, invece, l'ordine decide quando possiamo scegliere il testimone.

Sui naturali la formula $\forall x.\exists y.x < y$ è vera, perché dopo aver visto $x$ possiamo scegliere $y = x + 1$. La formula $\exists y.\forall x.x < y$ è invece falsa, perché chiede un solo $y$ che superi tutti i numeri, compreso sé stesso. Un verso vale comunque sempre: se c'è un testimone comune a tutti gli $x$, a maggior ragione ognuno ha il suo,

$$(\exists y.\forall x.P(x, y)) \Rightarrow (\forall x.\exists y.P(x, y)).$$

## 4.3 Regole di prova: individui particolari e generici

Le leggi ci dicono quali formule sono equivalenti. Per dimostrare qualcosa ci servono anche regole che dicano come usare e come ottenere un quantificatore.

Le due più semplici sono quasi ovvie. Se una proprietà vale per tutti, vale anche per un individuo particolare, e da $\forall x.P$ possiamo ricavare $P[t\,/\,x]$ per qualunque termine $t$. È l'**eliminazione dell'universale**. Se invece abbiamo un caso concreto che soddisfa la proprietà, sappiamo che un caso esiste, e da $P[t\,/\,x]$ ricaviamo $\exists x.P$, con l'**introduzione dell'esistenziale**. In entrambe le regole la sostituzione deve evitare catture.

L'esempio classico è quello di Socrate. La frase «tutti gli uomini sono mortali», applicata a lui, diventa «se Socrate è un uomo, allora è mortale». Poiché sappiamo che è un uomo, il modus ponens ci dà la conclusione.

### 4.3.1 Dimostrare una proprietà per tutti

Le altre due regole sono più delicate. Per dimostrare $\forall x.P$ non basta verificare la proprietà su un individuo che conosciamo, perché potrebbe avere qualcosa di speciale. Prendiamo invece un nome nuovo $d$, a cui non attribuiamo nessuna proprietà particolare, e dimostriamo $P[d\,/\,x]$.

Se il ragionamento non ha usato niente di specifico su $d$, allora funziona per chiunque, e possiamo concludere $\forall x.P$. È la **generalizzazione**. Perché sia lecita, il nome $d$ non deve comparire né nella formula da dimostrare né nelle ipotesi ancora in uso.

Per seguire queste condizioni ci serve un po' di vocabolario. Un'ipotesi è *aperta* finché il risultato dipende da essa, e la *chiudiamo*, o *scarichiamo*, quando una regola ci permette di toglierla dalle premesse. L'esempio che segue rende concreta questa distinzione.

### 4.3.2 Usare un oggetto di cui sappiamo l'esistenza

L'ultima regola riguarda le premesse esistenziali. Se sappiamo $\exists x.P$, sappiamo che un oggetto con quella proprietà c'è, ma non sappiamo quale. Possiamo allora dargli un nome nuovo $d$, assumere per il momento $P[d/x]$ e ragionare su di lui, a patto che la conclusione finale non nomini $d$.

Nel corso questo passo prende il nome di **skolemizzazione**, e qui lo presentiamo come la regola che usa un testimone esistenziale. Il nome deve essere nuovo anche rispetto alla conclusione. Non stiamo affermando che $(\exists x.P) \Rightarrow P[d/x]$ valga per una costante fissata in anticipo, il che sarebbe falso, ma soltanto che possiamo ragionare su un testimone di cui non conosciamo l'identità.

### 4.3.3 Una prova con un testimone temporaneo

Mettiamo insieme le regole in una prova completa. Supponiamo di sapere che ogni oggetto con la proprietà $P$ ha anche la proprietà $Q$, e che almeno un oggetto ha la proprietà $P$. Vogliamo dimostrare che almeno un oggetto ha la proprietà $Q$. Chiamiamo $\Gamma$ le due premesse, che resteranno disponibili per tutta la prova:

$$\Gamma = \{\forall x.(P(x) \Rightarrow Q(x)), \exists x.P(x)\}.$$

**1. Diamo un nome al testimone.** L'esistenziale non ci dice quale oggetto abbia la proprietà $P$. Lo chiamiamo $d$, con un nome mai usato prima, e apriamo l'ipotesi temporanea $H\colon P(d)$. Non stiamo dicendo che un oggetto già noto soddisfi $P$.

**2. Applichiamo la regola generale a quell'oggetto.** Dalla prima premessa, eliminando l'universale, ricaviamo $P(d) \Rightarrow Q(d)$. In questo momento le ipotesi aperte sono $\Gamma$ e $H$.

**3. Ricaviamo la proprietà cercata.** Con $H$ e il modus ponens otteniamo $Q(d)$, e introducendo l'esistenziale arriviamo a

$$\Gamma, H \vdash \exists x.Q(x).$$

**4. Chiudiamo l'ipotesi sul testimone.** La conclusione non nomina $d$, e non lo nomina nemmeno nessuna premessa di $\Gamma$. Il ragionamento funziona quindi per qualunque testimone garantito da $\exists x.P(x)$, e la regola di eliminazione dell'esistenziale ci permette di chiudere $H$:

$$\Gamma \vdash \exists x.Q(x).$$

La conclusione dipende ancora dalle due premesse iniziali, mentre l'ipotesi locale $P(d)$ è stata chiusa. Notiamo anche che cosa non possiamo concludere. La formula $\forall x.Q(x)$ non segue, perché $d$ era un oggetto con una proprietà particolare e non un individuo generico.

### 4.3.4 Che cosa garantisce un sistema di prova

Un **sistema di dimostrazione** è l'insieme delle regole di inferenza che possiamo usare. Ogni regola permette di passare da certe premesse a una conclusione, e una regola senza premesse si chiama assioma. Scriviamo $\Gamma \vdash P$ quando esiste una sequenza di passi consentiti che parte da $\Gamma$ e arriva a $P$. È una proprietà delle prove scritte, ben distinta da $\Gamma \vDash P$, che riguarda invece la verità nelle interpretazioni.

Le due nozioni sono collegate da due proprietà del sistema. Il sistema è **corretto** se tutto ciò che possiamo derivare è anche conseguenza logica delle premesse, ed è **completo** se possiamo derivare ogni conseguenza logica. Per il calcolo proposizionale valgono entrambe, e del sistema presentato nel corso useremo la correttezza. Per il primo ordine esistono sistemi completi, ma non esiste un procedimento che decida sempre, in un numero finito di passi, se una formula qualsiasi è valida.

### 4.3.5 Dall'ipotesi all'implicazione

Se assumiamo $H$ e ne ricaviamo $Q$, possiamo chiudere l'ipotesi e concludere $H \Rightarrow Q$, rispettando le condizioni delle regole sui quantificatori. È il contenuto del **teorema di deduzione**. Le altre premesse usate restano a disposizione, e il risultato continua a dipendere da esse.

Le ipotesi vanno tenute d'occhio anche quando sostituiamo sotto un quantificatore. Possiamo usare un'equivalenza, o le polarità del capitolo 2, soltanto se la relazione vale per tutti i valori su cui varia la variabile. Sapere che $P(x)$ è vera per il valore attuale di $x$ non ci autorizza a sostituirla con $T$ dentro $\forall x.P(x)$, perché il quantificatore considera anche tutti gli altri valori.

## 4.4 Uguaglianza e asserzioni sugli stati

Con il significato usuale dell'uguaglianza, $t = u$ è vera quando i due termini indicano lo stesso elemento. Da qui discende la **legge di Leibniz**, secondo cui sostituire un individuo con uno uguale non cambia le sue proprietà:

$$x = y \Rightarrow (P \equiv P[y/x]).$$

Come sempre la sostituzione riguarda le occorrenze libere e non deve catturare variabili. In pratica, da $x = y$ e da $P$ possiamo concludere $P[y/x]$, e per esempio da $x = y$ e $x > 0$ ricaviamo $y > 0$.

Le **leggi del singoletto** dicono che quantificare su un unico individuo equivale a sostituirlo. Per due variabili distinte $x$ e $y$, e sempre evitando catture,

$$(\forall x.(x = y \Rightarrow P)) \equiv P[y/x],$$

$$(\exists x.(x = y \wedge P)) \equiv P[y/x].$$

La seconda legge si capisce bene ragionando sul testimone. L'unico valore che può soddisfare $x = y$ è quello di $y$, quindi l'esistenziale è vero se quel valore soddisfa $P$ e falso altrimenti, perché nessun altro valore può prenderne il posto. La prima legge controlla lo stesso unico individuo attraverso un'implicazione.

Abbiamo ora tutto ciò che serve per parlare dei programmi. Un'asserzione come $x \geq 0 \wedge y > 0$ è una formula aperta, e uno stato fornisce i valori delle variabili di programma esattamente come un assegnamento. Scriviamo $\sigma \vDash P$ se $P$ è vera con quei valori nell'interpretazione numerica usuale. Nei prossimi capitoli useremo coppie di asserzioni come queste per distinguere ciò che chiediamo prima di un programma da ciò che vogliamo ottenere dopo.

## 4.5 Per proseguire: leggi sui domini e indebolimento

Quando il filtro di un quantificatore ristretto è un'unione di due gruppi, la proprietà va verificata su entrambi, e per trovare un testimone basta cercarlo in uno dei due:

$$(\forall x\colon P \vee Q.R) \equiv ((\forall x\colon P.R) \wedge (\forall x\colon Q.R)),$$

$$(\exists x\colon P \vee Q.R) \equiv ((\exists x\colon P.R) \vee (\exists x\colon Q.R)).$$

Anche le leggi distributive e quelle di De Morgan viste prima valgono con uno stesso filtro ristretto.

Infine, l'indebolimento funziona nella direzione consueta anche sotto i quantificatori. Valgono $(\forall x.P) \Rightarrow \forall x.(P \vee Q)$ e $(\exists x.P) \Rightarrow \exists x.(P \vee Q)$, e da una formula quantificata su $P \wedge Q$ possiamo sempre tenere soltanto $P$. I quantificatori non invertono la polarità, e quindi una relazione che vale per tutti gli individui si può applicare dentro le loro portate.

---

### Verifica 4

**1.** Nega $\forall x.(p(x) \Rightarrow \exists y.q(x, y))$, portando la negazione fino agli atomi. Spiega che cosa dovresti trovare per mostrare falsa la formula iniziale.

**2.** Su $D = \{a, b\}$, la relazione $R$ è vera soltanto per $(a, b)$ e $(b, a)$. Valuta $\forall x.\exists y.R(x, y)$ ed $\exists y.\forall x.R(x, y)$. Mostra poi con un esempio perché da $P(d)$ per una costante già nota non si può sempre generalizzare a $\forall x.P(x)$.

# Stati, espressioni e comandi

**In breve**
Nel primo capitolo abbiamo seguito un programma a occhio, guardando i valori che cambiavano. Ora è il momento di dire con precisione che cosa fa ciascun comando. Vedremo come si calcola un'espressione, come un assegnamento modifica lo stato e come scelte e ripetizioni decidono la strada dell'esecuzione, scoprendo anche che non sempre un programma arriva a un risultato.

Per capire un programma dobbiamo distinguere due operazioni diverse, leggere un valore e modificarlo. Nell'espressione `x + 1` leggiamo il valore di `x` e calcoliamo un numero, senza cambiare niente. Nel comando `x := x + 1` usiamo quel numero per aggiornare `x`. Per questo usiamo due segni diversi, `:=` per l'azione di assegnare e = per l'uguaglianza che compare nelle asserzioni.

## 5.1 Lo stato e la valutazione delle espressioni

Uno stato $\sigma$ associa un valore a ciascun identificatore del programma, per esempio $\sigma(x) = 10$ e $\sigma(y) = 3$. Nel linguaggio che studiamo i valori sono interi oppure booleani, cioè Val=$\mathbb{Z} \cup$ Bool, dove i due valori booleani sono vero e falso. Gli interi sono quelli della matematica, senza limiti di memoria e senza i fenomeni di traboccamento dei calcolatori reali.

Uno stato può anche essere parziale e non dare un valore a tutti i nomi possibili. Nel seguito supporremo però che ogni identificatore letto dal programma sia presente nello stato e abbia il tipo dichiarato, e questa ipotesi ci permetterà di dire che leggere una variabile è sempre un'operazione definita.

### 5.1.1 Espressioni e tipi

Le espressioni si costruiscono a partire dalle costanti intere, dai valori booleani `true` e `false` e dai nomi delle variabili, e si combinano con parentesi e operatori. I nomi `true` e `false` sono riservati ai due valori booleani. Gli operatori `+`, `-`, `*`, `div` e `mod` lavorano sugli interi, mentre i confronti e gli operatori logici producono booleani.

Valgono le precedenze usuali, che distinguono `8 + 5 * 3`, che vale 23, da `(8 + 5) * 3`, che vale 39. Quando c'è un dubbio, le parentesi rendono visibile quale operazione viene prima.

Prima di calcolare un'espressione ne controlliamo i tipi. L'espressione `(x + 5) and (x > y)` può rispettare la grammatica, ma `and` vuole due booleani e `x + 5` è un intero, quindi l'espressione va rifiutata. Questo controllo si chiama **semantica statica**, perché analizza la forma del programma senza eseguirlo. Da qui in poi considereremo soltanto espressioni con i tipi corretti.

### 5.1.2 Calcolare il valore

Scriviamo $\mathcal{E}(E, \sigma)$ per il valore dell'espressione $E$ nello stato $\sigma$. Per calcolare `(x + 1) * y` nello stato dell'esempio leggiamo $\sigma(x) = 10$, otteniamo $10 + 1 = 11$, leggiamo $\sigma(y) = 3$ e infine moltiplichiamo, ottenendo $11 \cdot 3 = 33$. Durante tutto il calcolo lo stato non cambia.

La definizione generale segue la costruzione dell'espressione, esattamente come facevamo con le formule:

$$\mathcal{E}(x,\sigma)=\sigma(x), \qquad \mathcal{E}(E+E',\sigma)=\mathcal{E}(E,\sigma)+\mathcal{E}(E',\sigma).$$

Le costanti indicano i valori corrispondenti, quindi `true` indica vero e `false` falso. Gli altri operatori funzionano come la somma, applicando l'operazione ai valori degli argomenti, mentre le parentesi non cambiano il valore e `not` nega un booleano. Precisiamo che `and` e `or` valutano sempre entrambi gli argomenti, senza il «cortocircuito» che si trova in alcuni linguaggi reali.

### 5.1.3 Un'espressione può non avere valore

Il controllo dei tipi evita molti errori, ma non tutti. L'espressione `x div (y - 3)` riceve correttamente due interi, eppure con $y=3$ chiede una divisione per zero. Per alcune coppie di espressione e stato il valore semplicemente non esiste, e per questo diciamo che $\mathcal{E}$ è una **funzione parziale**. Il valore mancante si indica con $\perp$.

Conviene allora precisare anche la divisione intera. Se $a \geq 0$ e $b > 0$, `a div b` è il quoziente e `a mod b` è il resto, cioè il numero non negativo minore di $b$ che avanza. Per esempio

$$17 \text{ div } 5 = 3, \qquad 17 \text{ mod } 5 = 2.$$

Tutti gli esempi del libro usano numeri con questi segni. Per dare comunque un significato anche agli altri casi, fissiamo per ogni intero $a$ e ogni $b \neq 0$ la convenzione

$$a \text{ div } b = \lfloor a/b \rfloor, \quad a \text{ mod } b = a - b\lfloor a/b \rfloor,$$

dove $\lfloor u \rfloor$ indica il più grande intero che non supera $u$. È una scelta che rende precisa la semantica del nostro piccolo linguaggio, e altri linguaggi reali ne fanno di diverse sui numeri negativi.

## 5.2 Assegnamento, sequenza e perdita di informazione

Per descrivere l'effetto di un assegnamento ci serve una notazione. Scriviamo $\sigma[v/x]$ per lo stato che coincide con $\sigma$ su tutti i nomi tranne $x$, al quale associa il valore $v$. Il comando `x := E` prima calcola tutta l'espressione $E$ nello stato vecchio, e solo dopo aggiorna $x$. Partendo da $x=10, y=3$, per esempio, il comando `x := x + y` porta a $x=13, y=3$.

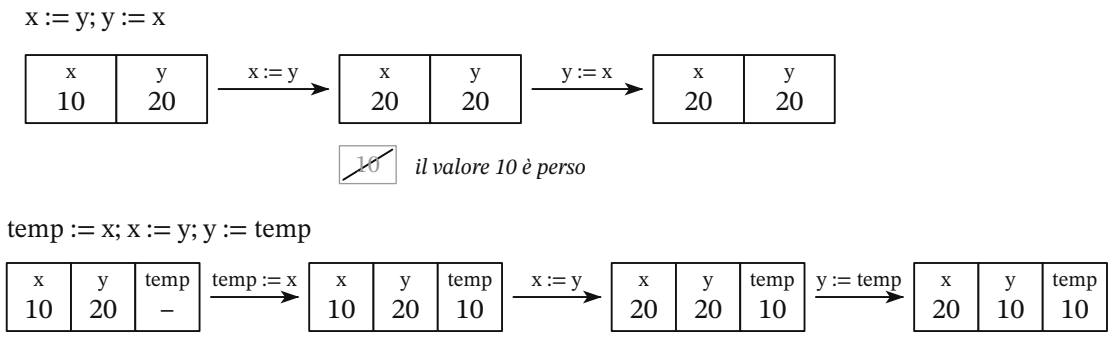
> **Concetto chiave · Espressioni e comandi**
> Un'espressione legge lo stato e calcola un valore. Un comando viene eseguito e, se termina con successo, produce uno stato. Un assegnamento sostituisce il vecchio valore della variabile destinazione.

Una sequenza `C; D` esegue prima `C` e poi `D` nello stato che `C` ha prodotto. Sembra ovvio, ma ha conseguenze che sorprendono chi inizia. Proviamo a scambiare due variabili scrivendo `x := y; y := x`, partendo da $x=10, y=20$. Il primo comando porta a $x=20, y=20$, e il secondo legge il nuovo valore di $x$, che è 20, lasciando ancora $y=20$. Il vecchio 10 è andato perso al primo passo, e nessun comando successivo può ritrovarlo.

Per non perderlo dobbiamo copiarlo prima da qualche parte, in una variabile d'appoggio.

```
temp := x;
x := y;
y := temp
```

Dopo il primo comando `temp` contiene 10, dopo il secondo `x` contiene 20 e dopo il terzo `y` riprende il 10 messo da parte. La variabile d'appoggio è necessaria proprio perché ogni comando legge lo stato del momento, e non quello iniziale.




Lo scambio mette da parte il valore che il secondo assegnamento sovrascriverà. Le frecce seguono la direzione dell'esecuzione.

## 5.3 Scelte e ripetizioni: if e while

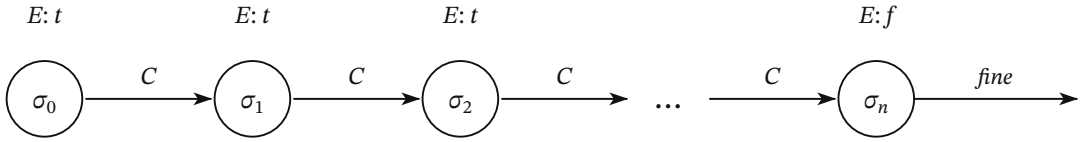
Il comando più semplice di tutti è `skip`, che non fa nulla e lascia lo stato com'era. Il condizionale `if E then C else D fi` serve invece a scegliere. Valuta la guardia booleana $E$ ed esegue soltanto $C$ se è vera, soltanto $D$ se è falsa. Se la guardia non ha un valore, il condizionale non può scegliere nessun ramo.

Per mettere in `r` il valore assoluto di `x` possiamo scrivere

```
if x > 0 then r := x else r := -x fi
```

Con $x = 7$ la guardia è vera e otteniamo $r = 7$, con $x = -7$ scegliamo il secondo ramo e otteniamo di nuovo $r = 7$. Con $x = 0$ la guardia è falsa e il secondo ramo assegna $-0 = 0$, che è ancora il risultato giusto. In ogni caso la variabile `x` non cambia.

Il ciclo `while E do C endw` controlla la guardia prima di ogni possibile esecuzione del corpo $C$. Se la guardia è falsa esce subito, se è vera esegue il corpo e torna a controllarla nello stato nuovo. Il numero di ripetizioni non è deciso in anticipo, ma dipende da come evolve lo stato.




Ogni freccia rappresenta un'intera esecuzione del corpo. Il controllo della guardia avviene anche nello stato iniziale; il primo controllo falso conclude il ciclo.

Vediamolo con il calcolo di una potenza, partendo da base=2 ed esp=3.

```
ris := 1;
while esp > 0 do
    ris := ris * base;
    esp := esp - 1
endw
```

Al primo controllo della guardia la coppia $(\text{ris}, \text{esp})$ vale $(1, 3)$. Ogni giro moltiplica il risultato per 2 e toglie 1 all'esponente che rimane, quindi i controlli successivi incontrano $(2, 2)$, $(4, 1)$ e $(8, 0)$. A quel punto la guardia è falsa e il risultato è 8. Se l'esponente iniziale fosse zero il corpo non verrebbe mai eseguito e `ris` varrebbe 1, come vuole la regola $A^0 = 1$.

Questa traccia ci convince sull'esempio, ma non dimostra che il programma funzioni per ogni esponente. Per farlo cercheremo una relazione che leghi sempre il prodotto già calcolato al lavoro ancora da fare, cioè un invariante come quello del capitolo 1.

## 5.4 La semantica dei comandi e le esecuzioni senza risultato

Come per le espressioni, possiamo descrivere i comandi con una funzione. Scriviamo $\mathcal{C}(C,\sigma)$ per lo stato finale che il comando $C$ produce partendo da $\sigma$. Per i primi costrutti la definizione è

$$\mathcal{C}(\text{skip},\sigma)=\sigma,$$

$$\mathcal{C}(\text{x := E},\sigma)=\sigma[\mathcal{E}(E,\sigma)/x],$$

$$\mathcal{C}(C;D,\sigma)=\mathcal{C}(D,\mathcal{C}(C,\sigma)).$$

L'ultima formula rispecchia l'ordine dei comandi, perché lo stato prodotto da $C$ diventa il punto di partenza di $D$. Per il condizionale si sceglie $\mathcal{C}(C,\sigma)$ oppure $\mathcal{C}(D,\sigma)$ a seconda del valore della guardia.

Il ciclo $W \equiv$ while E do C endw richiede due casi. Se la guardia è falsa il corpo non viene eseguito e lo stato resta quello iniziale,

$$\mathcal{C}(W,\sigma)=\sigma.$$

Se la guardia è vera si esegue prima $C$, e poi si ricomincia lo stesso ciclo nello stato ottenuto,

$$\mathcal{C}(W,\sigma)=\mathcal{C}(W,\mathcal{C}(C,\sigma)).$$

Questa seconda clausola richiama il ciclo stesso. Descrive il risultato quando tutte le valutazioni necessarie sono definite e prima o poi si incontra un controllo falso.

### 5.4.1 Quando un programma non arriva a un risultato

Anche $\mathcal{C}$, come $\mathcal{E}$, è una funzione parziale, perché non tutte le esecuzioni producono uno stato finale. Le ragioni possono essere due. Può verificarsi un errore, come in `x := 1 div 0`, oppure il calcolo può andare avanti per sempre. Consideriamo il ciclo

```
while x >= y do x := x + 1 endw
```

Se all'inizio $x \geq y$, ogni giro fa crescere ancora $x$, la guardia resta vera per sempre e il ciclo non termina. Se invece all'inizio $x < y$, la guardia è subito falsa e si esce senza eseguire il corpo.

Basta cambiare un segno per cambiare tutto. Con `x := x - 1` al posto dell'incremento il ciclo termina per ogni coppia di interi, perché $y$ resta fisso e $x$ scende di uno alla volta finché diventa minore di $y$. Per dimostrarlo in generale ci servirà una misura del lavoro che resta da fare, che introdurremo nel capitolo 7.

### 5.4.2 Tre modi di descrivere un programma

Ci sono tre modi diversi, e complementari, di descrivere il significato di un programma. Quando seguiamo i singoli passi dell'esecuzione usiamo la **semantica operazionale**. Quando associamo a ogni comando la funzione parziale che porta dallo stato iniziale a quello finale, come abbiamo appena fatto con $\mathcal{C}$, usiamo la **semantica denotazionale**. Nel prossimo capitolo descriveremo invece il comportamento con proprietà vere prima e dopo i comandi, e le regole che collegano queste proprietà formano la **semantica assiomatica**.

---

**Verifica 5**

**1.** Esegui `x := x + y; y := x - y; x := x - y` da $x=3, y=7$. Poi usa valori iniziali generici $A, B$ per spiegare il comportamento per tutti gli interi.

**2.** Per `while x ≠ 0 do x := x - 2 endw`, confronta le esecuzioni da $x=4$ e da $x=5$. Che cosa mostra l'esempio sulla necessità di controllare la terminazione?

# Triple di Hoare e invarianti dei cicli

**In breve**
Seguire un programma su un esempio ci convince, ma non ci garantisce nulla sugli altri ingressi. Le triple di Hoare permettono di fare il salto, collegando con una formula ciò che chiediamo prima del programma a ciò che vogliamo ottenere dopo. In questo capitolo vedremo le regole per ciascun comando, scoprendo che conviene spesso ragionare all'indietro, e arriveremo all'invariante, lo strumento che rende possibile ragionare sui cicli.

Con $x = 17$ e $y = 5$ possiamo seguire una divisione passo per passo e ottenere quoziente 3 e resto 2. Ma un programma serve perché lo useremo su dati che non abbiamo provato, e per fidarci di lui vogliamo qualcosa di più di un esempio, cioè una prova che copra tutti gli ingressi ammessi.

## 6.1 Precondizione, postcondizione e correttezza

Un'**asserzione** è una formula che parla delle variabili del programma, e scriviamo $\sigma \vDash P$ per dire che $P$ è vera nello stato $\sigma$. Se per esempio $\sigma(x) = 18$ e $\sigma(z) = -8$, la formula $x > 0 \land z < 0$ diventa $18 > 0 \land -8 < 0$, che è vera. La formula ottenuta mettendo al posto delle variabili i valori dello stato si indica con $P^\sigma$.

Il valore di verità di un'asserzione dipende soltanto dalle sue variabili libere, quindi cambiare un'altra variabile non può modificarlo. Lo stesso vale per le espressioni, il cui valore dipende soltanto dai nomi che leggono. Con le asserzioni possiamo ora scrivere che cosa promette un programma.

**Concetto chiave · Tripla di Hoare**
La tripla $\{P\}\ C\ \{Q\}$ collega una **precondizione** $P$, un comando $C$ e una **postcondizione** $Q$.

In **correttezza totale**, ogni esecuzione da uno stato che soddisfa $P$ termina con successo e soddisfa $Q$ all'uscita.

In **correttezza parziale**, da uno stato che soddisfa $P$, ogni esecuzione che arriva a uno stato finale soddisfa $Q$.

La correttezza parziale è una promessa più debole, perché non garantisce che uno stato finale esista. Nella convenzione che adottiamo, né un errore né un'esecuzione infinita producono uno stato finale, e quindi una prova soltanto parziale non esclude nemmeno gli errori. Per questo, quando vogliamo un'esecuzione che vada a buon fine, controlleremo sempre esplicitamente che le espressioni abbiano un valore. Se serve distinguere le due letture scriveremo $\vDash_{tot}$ oppure $\vDash_{par}$.

Due esempi aiutano a fissare le idee. La tripla $\{x > 1\}$ x := x + 1 $\{x > 2\}$ è totalmente corretta, perché il nuovo valore di $x$ è il vecchio più uno e quindi supera 2. La tripla $\{x = 5\}$ x := x + 1 $\{x = 5\}$ è invece falsa, perché l'unico valore finale possibile di $x$ è 6.

Una tripla si può usare in tre modi. Possiamo cercare $Q$ per descrivere che cosa fa un comando, cercare $C$ per costruire un programma che soddisfi una richiesta, oppure avere già tutti e tre gli elementi e verificare che la proposta funzioni.

### 6.1.1 Il valore finale e quello iniziale

Una postcondizione si legge sempre nello stato finale, e questo può giocare brutti scherzi. La specifica $\{x>0\}\ C\ \{y=2x\}$ sembra chiedere il doppio dell'ingresso, ma è soddisfatta anche dal programma `y := 10; x := 5`, che cambia $x$ a piacere.

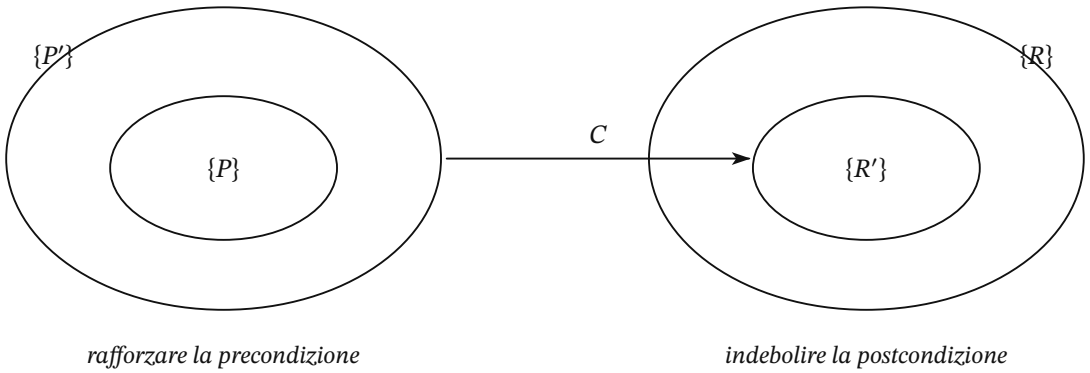
Se vogliamo davvero il doppio del valore iniziale, dobbiamo fissarlo con una variabile di specifica e scrivere $\{x=A\wedge A>0\}\ C\ \{y=2A\}$. La lettera $A$ non compare nei comandi e resta la stessa per tutta la prova, quindi ricorda il valore di partenza anche se il programma modifica $x$.

## 6.2 Conseguenza e definitezza delle espressioni

Se un programma funziona su tutti gli stati con $x>0$, funziona in particolare su quelli con $x>10$, che sono una parte dei primi. Questo è il **rafforzamento della precondizione**. Allo stesso modo, se un programma garantisce $x=5$, garantisce anche $x>0$, che è una richiesta più modesta, e si parla di **indebolimento della postcondizione**. La regola di conseguenza riunisce le due osservazioni:

$$\frac{P\Rightarrow P'\quad \{P'\}C\{Q'\}\quad Q'\Rightarrow Q}{\{P\}C\{Q\}}.$$

Le implicazioni devono valere in tutti gli stati. Pensando agli insiemi di stati, quelli che soddisfano $P$ sono contenuti in quelli che soddisfano $P$', e quelli che soddisfano $Q$' sono contenuti in quelli che soddisfano $Q$. La regola permette quindi di restringere gli ingressi e di accontentarsi di conclusioni più deboli, mai il contrario.




Una precondizione più forte restringe i punti di partenza; una postcondizione più debole amplia i risultati ammessi.

### 6.2.1 Quando un'espressione ha valore

Per parlare di esecuzioni che vanno a buon fine ci serve un modo di dire che un'espressione $E$ ha un valore, e lo facciamo con l'asserzione $\mathrm{def}(E)$. Le costanti e le variabili presenti nello stato sono sempre definite. Somme, prodotti, differenze, confronti e operatori booleani sono definiti quando lo sono i loro argomenti. Per la divisione e il resto serve in più che il divisore non sia nullo,

$$\mathrm{def}(E\ \mathrm{div}\ D)\equiv \mathrm{def}(E)\wedge\mathrm{def}(D)\wedge D\neq 0,$$

e la stessa clausola vale per `mod`. Per la negazione e le parentesi basta che sia definita l'espressione interna.

Per esempio, $\mathrm{def}(x\ \mathrm{div}\ (z-y))$ equivale a $z\neq y$, perché la sottrazione è sempre definita ma il suo risultato non deve essere zero. E siccome i nostri operatori booleani non usano il cortocircuito, l'espressione `x ≠ 0 and y div x > 1` richiede comunque $x\neq 0$ per avere un valore, anche se la prima parte sembrerebbe proteggere la seconda.

## 6.3 Assegnamento: ragionare a partire dal risultato

Supponiamo di voler garantire $x > y$ dopo il comando `x := x + 2`. Il valore finale di $x$ sarà il vecchio valore aumentato di 2, quindi prima del comando dobbiamo chiedere che sia $x + 2 > y$. Abbiamo ottenuto la condizione iniziale partendo da quella finale e mettendo al posto di $x$ il valore che le verrà assegnato.

In generale scriviamo $Q[E/x]$ per la formula che si ottiene sostituendo $E$ alle occorrenze libere di $x$ in $Q$, rinominando prima, se serve, le variabili quantificate che causerebbero catture. Quando $E$ è definita vale una proprietà fondamentale,

$$\sigma[\mathcal{E}(E,\sigma)/x] \vDash Q \text{ se e solo se } \sigma \vDash Q[E/x].$$

A sinistra valutiamo $Q$ dopo l'aggiornamento, a destra anticipiamo nella formula il valore che verrà assegnato, e le due cose coincidono. Da qui nasce l'assioma dell'assegnamento:

$$\{\mathrm{def}(E) \wedge Q[E/x]\} \ \ \mathrm{x} := \mathrm{E} \ \ \{Q\}.$$

Le due parti della precondizione hanno compiti diversi. La definitezza assicura che il comando produca uno stato, mentre la sostituzione assicura che quello stato soddisfi $Q$. Se la precondizione che ci viene data è un'altra formula $P$, basta dimostrare $P \Rightarrow \mathrm{def}(E) \wedge Q[E/x]$ e poi usare la regola di conseguenza.

Qualche caso particolare aiuta a prendere confidenza con l'assioma. Per `x := 5` con postcondizione $x = 5$ la sostituzione dà $5 = 5$, che è sempre vera, e quindi non serve chiedere nulla sul vecchio valore di $x$. Per `x := 1 div y` serve invece $y \neq 0$ anche se la postcondizione non parla di $x$, perché in correttezza totale dobbiamo comunque escludere l'errore. Infine il comando `skip` ha l'assioma $\{P\}$ skip $\{P\}$, dato che non cambia nessun valore e quindi ogni asserzione vera prima rimane vera dopo.

## 6.4 Sequenza: trovare l'asserzione intermedia

Per dimostrare $\{P\} \ \ C; D \ \ \{Q\}$ cerchiamo un'asserzione intermedia $R$ per cui valgano sia $\{P\}\,C\,\{R\}$ sia $\{R\}\,D\,\{Q\}$. In altre parole il primo comando deve consegnare al secondo uno stato che si trova nelle condizioni di cui il secondo ha bisogno. È la regola della sequenza, e il lavoro sta tutto nel trovare $R$.

Torniamo allo scambio di due variabili, con ingresso $x = N \wedge y = M$ e uscita $x = M \wedge y = N$. Il programma avrà la forma `t := E; x := y; y := t`, e la domanda è quale valore debba mettere da parte il primo comando. Invece di tirare a indovinare, risaliamo dall'ultimo comando.

Dopo `y := t` vogliamo $y = N$, quindi prima di quel comando ci serve $t = N$, mentre la richiesta su $x$ rimane la stessa, perché il comando non tocca $x$. Risaliamo poi attraverso `x := y`. Per avere $x = M$ dopo questo passo, prima deve valere $y = M$. La tabella riassume le condizioni che troviamo davanti a ciascun comando.

| Comando | Condizione prima del comando |
|---|---|
| y := t | $x = M \wedge t = N$ |
| x := y | $y = M \wedge t = N$ |
| t := E | $y = M \wedge E = N$ |

Resta da scegliere $E$, che deve essere definita e valere $N$. L'ingresso ci dice che $x = N$, quindi scegliamo $E = x$. Notiamo che il programma legge la variabile `x`, mentre $N$ serve soltanto alla prova e non può comparire nei comandi.

Con $N = 3$ e $M = 7$ l'esecuzione va da $(x, y) = (3, 7)$ a $(7, 3)$, passando per $t = 3$. È curioso osservare che l'esecuzione procede dall'alto verso il basso, mentre la ricerca delle condizioni è partita dall'ultimo comando e ha risalito il programma.

Un esempio più breve mostra un'altra libertà della regola. Per il programma `x := x + 1; y := y - 1` con risultato $x > y$, prima dell'ultimo comando serve $x > y - 1$ e prima del primo serve $x + 1 > y - 1$. Se la precondizione data è $x \geq y - 1$, va bene lo stesso, perché implica quella calcolata: da $x \geq y - 1$ segue $x + 1 \geq y > y - 1$. La regola non chiede che la precondizione sia esattamente quella trovata all'indietro, ma soltanto che la implichi.

## 6.5 Condizionale: utilizzare l'informazione della guardia

Per verificare `if E then C else D fi` con ingresso $P$ e uscita $Q$ controlliamo prima che la guardia abbia un valore, cioè che $P \Rightarrow \mathrm{def}(E)$. Poi dimostriamo separatamente i due rami, $\{P \wedge E\}\, C\, \{Q\}$ e $\{P \wedge \neg E\}\, D\, \{Q\}$. Ogni ramo riceve così in regalo l'informazione che ha portato a sceglierlo.

Vogliamo per esempio che $m$ contenga il massimo fra $x$ e $y$, e proponiamo

```
if x < y then m := y else m := x fi
```

La guardia è sempre definita. Nel primo ramo sappiamo che $x < y$, quindi assegnare $y$ a $m$ dà il massimo. Nel secondo sappiamo che $x \geq y$, e assegnare $x$ dà di nuovo il massimo. Il vecchio valore di $m$ non conta, perché entrambi i rami lo sostituiscono.

### 6.5.1 Una guardia che richiede una divisione

La definitezza della guardia diventa interessante quando contiene una divisione, come in

```
if x div y > 0 then m := x else m := y fi
```

L'ipotesi $x \geq 0 \wedge y \geq 0$ non basta, perché permette $y = 0$, e con $y = 0$ la guardia non ha valore. Chiediamo quindi $x \geq 0 \wedge y > 0$.

Con questi vincoli la guardia $x$ div $y > 0$ equivale a $x \geq y$, perché il quoziente è almeno 1 esattamente quando $y$ sta almeno una volta in $x$. Nel ramo vero scegliamo quindi il massimo $x$, e nel ramo falso il massimo $y$. L'ipotesi sul divisore ha dunque due ruoli, perché rende possibile valutare la guardia e ci permette di leggerla come un confronto.

## 6.6 Invariante e correttezza parziale del while

Arriviamo ai cicli, con la divisione fatta per sottrazioni successive. Supponiamo $x \geq 0$ e $y > 0$, e chiediamo che $x$ e $y$ non vengano modificati.

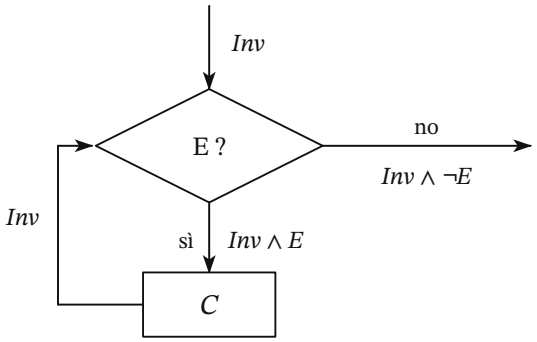
```
q := 0; r := x;
while r >= y do
    q := q + 1;
    r := r - y
endw
```

Partendo da $x = 17, y = 5$, ai controlli della guardia troviamo le coppie $(q, r) = (0, 17), (1, 12), (2, 7), (3, 2)$. Che cosa hanno in comune queste righe? A ogni giro la quantità $q\,y$ cresce di 5 e il resto $r$ cala di 5, quindi la loro somma non cambia e resta sempre uguale a $x$. Inoltre $r$ non diventa mai negativo, perché il corpo sottrae $y$ soltanto quando $r \geq y$.

Queste osservazioni suggeriscono di scegliere

$$I \equiv (x = q\,y + r \wedge r \geq 0 \wedge y > 0).$$

È un **invariante**, cioè una proprietà che vale dopo l'inizializzazione e viene ristabilita dopo ogni esecuzione completa del corpo. Durante il corpo può anche smettere di valere per un momento. Subito dopo l'incremento di $q$, per esempio, l'uguaglianza è falsa, e torna vera soltanto dopo la sottrazione da $r$. Ciò che conta è che sia vera ogni volta che controlliamo la guardia.




La proprietà deve valere prima del controllo e dopo l'intero corpo. All'uscita aggiungiamo l'informazione che la guardia è falsa.

### 6.6.1 Perché l'invariante vale anche all'uscita

La regola del ciclo, in correttezza parziale, è

$$\frac{\{I \wedge E\}\ C\ \{I\}}{\{I\}\ \text{while E do C endw}\ \{I \wedge \neg E\}}.$$

Il motivo per cui funziona è un ragionamento a catena. Prima della prima iterazione vale $I$. Se vale prima di un giro che viene eseguito, allora anche la guardia è vera, e la premessa della regola ci dice che dopo il corpo $I$ vale di nuovo. Ripetendo questo passaggio per tutti i giri effettivamente completati, scopriamo che $I$ vale a ogni controllo raggiunto. Quando il ciclo esce la guardia è falsa, e quindi nello stato finale abbiamo $I \wedge \neg E$. È un ragionamento per induzione sul numero dei giri, che funziona senza bisogno di sapere in anticipo quanti saranno.

Per una specifica completa servono ancora l'ingresso e l'uscita, cioè $P \Rightarrow I$ e $I \wedge \neg E \Rightarrow Q$. Nella divisione la guardia falsa dice $r < y$, e insieme all'invariante otteniamo $x = q\,y + r \wedge 0 \le r < y$, che è esattamente la definizione di quoziente e resto. Questa prova descrive però soltanto le esecuzioni che arrivano all'uscita. Che l'uscita venga davvero raggiunta, e senza errori, è il tema del prossimo capitolo.

---

**VERIFICA 6**

**1.** Calcola una precondizione che garantisca $y > 10$ dopo `y := x * 2`. Verifica poi se $x \ge 6$ è sufficiente. Motiva con l'assioma, senza limitarti a provare alcuni valori.

**2.** La precondizione $z = 5 \wedge x = 3$ garantisce $z > 4$ dopo `x := 0; y := z div x` in correttezza totale? Calcola all'indietro il controllo di definitezza e spiega la risposta.

# Correttezza totale e funzioni di terminazione

**In breve**
Un invariante ci dice che cosa otteniamo se il ciclo finisce, ma tace sul fatto che finisca davvero. Per colmare questa lacuna basta un'idea semplice: trovare una quantità che parte da un numero naturale e scende a ogni giro, e che quindi non può scendere per sempre. Con questa idea completeremo la prova della divisione, impareremo a costruire un invariante per una somma e chiuderemo con un problema geometrico in cui la quantità giusta è tutt'altro che ovvia.

Nella divisione per sottrazioni ogni giro toglie qualcosa al resto, e già questo ci fa intuire che il ciclo prima o poi debba fermarsi. Per trasformare l'intuizione in una prova dobbiamo dire con precisione che cosa diminuisce e perché non può diminuire all'infinito. L'invariante descrive la relazione che rimane vera durante il calcolo, mentre la quantità che cerchiamo ora, la funzione di terminazione, misura il cammino che resta verso l'uscita.

## 7.1 Una quantità naturale che diminuisce

Con $x = 17$ e $y = 5$ il resto vale $17, 12, 7, 2$ ai successivi controlli della guardia. È sempre un intero non negativo, e diminuisce strettamente a ogni giro eseguito. Una sequenza di questo tipo non può essere infinita, perché partendo da un naturale $k$ ogni diminuzione fra interi toglie almeno 1, e dopo $k$ diminuzioni non ci sarebbe più spazio senza scendere sotto zero.

**Concetto chiave · Funzione di terminazione**
Una funzione di terminazione associa agli stati considerati una quantità $t$ intera, definita e non negativa. Se ogni iterazione completata la fa diminuire strettamente, e ogni iterazione termina con successo, il ciclo non può eseguire infinite iterazioni.

### 7.1.1 Perché chiediamo un numero naturale

La sola diminuzione non basterebbe. La successione di interi $0, -1, -2, \ldots$ diminuisce a ogni passo senza fermarsi mai, perché manca un limite inferiore. Nemmeno chiedere un numero reale positivo sarebbe sufficiente, dato che $1, 1/2, 1/4, \ldots$ è una successione infinita e strettamente decrescente di numeri positivi. Il fatto di lavorare con i naturali, dove i passi hanno almeno ampiezza 1 e c'è un fondo, è quindi una parte essenziale dell'argomento.

### 7.1.2 Confrontare il valore vecchio e quello nuovo

Per dire che $t$ diminuisce dobbiamo confrontare il suo valore prima del corpo con quello dopo, e i programmi non ricordano i valori vecchi. La soluzione è la stessa che abbiamo usato per gli ingressi, cioè una variabile di specifica nuova $V$, che fotografa il valore di partenza e che nessun comando può modificare. Il progresso si scrive allora

$$\{I \wedge E \wedge t = V\}\ C\ \{t < V\}.$$

La tripla non chiede al programma di mantenere $t$. Permette soltanto alla prova di confrontare il nuovo valore di $t$ con quello vecchio, rimasto fissato in $V$.

## 7.2 La regola del while in correttezza totale

Una prova di correttezza totale deve garantire tre cose, cioè che ogni controllo della guardia abbia un valore, che ogni esecuzione del corpo termini senza errori e che le iterazioni siano in numero finito. Per un ciclo $W$ della forma `while E do C endw` e una specifica $\{P\}\ W\ \{Q\}$, queste garanzie diventano cinque obblighi di prova.

1. **Ingresso.** $P \Rightarrow I \wedge \mathrm{def}(E)$: prima del primo controllo valgono l'invariante e la definitezza della guardia.
2. **Preservazione.** $\{I \wedge E\}\, C\, \{I \wedge \mathrm{def}(E)\}$ in correttezza totale: il corpo termina con successo e prepara il controllo successivo.
3. **Limite inferiore.** $I \Rightarrow t \geq 0$, con $t$ intera e definita negli stati di $I$.
4. **Progresso.** $\{I \wedge E \wedge t = V\}\, C\, \{t < V\}$ in correttezza totale, per ogni valore fissato di $V$.
5. **Uscita.** $I \wedge \neg E \Rightarrow Q$: quando il ciclo si ferma, le informazioni rimaste danno il risultato richiesto.

Gli obblighi si incastrano fra loro in modo naturale. La preservazione assicura che il ragionamento possa ripartire a ogni controllo, con l'invariante vero e la guardia definita. Se la guardia restasse vera per sempre otterremmo infinite diminuzioni strette fra numeri naturali, il che è impossibile, quindi prima o poi incontriamo un controllo falso. A quel punto l'invariante e l'obbligo di uscita ci danno $Q$.

Quando il ciclo è preceduto da un'inizializzazione, come quasi sempre accade, il primo obbligo diventa una tripla sul comando iniziale invece di un'implicazione diretta.

## 7.3 Divisione intera: completare la prova

Riprendiamo la divisione, con l'inizializzazione `q := 0; r := x`, la guardia $r \geq y$ e il corpo `q := q + 1; r := r - y`. L'invariante è $I \equiv (x = q\,y + r \wedge r \geq 0 \wedge y > 0)$, e come funzione di terminazione scegliamo il resto, $t = r$. La condizione $y > 0$ fa parte dell'invariante perché ci servirà a ogni giro per dimostrare il progresso.

### 7.3.1 Inizializzazione e preservazione

Dopo l'inizializzazione abbiamo $x = 0 \cdot y + x$ e $r = x \geq 0$, quindi l'invariante vale. La guardia confronta soltanto due variabili ed è sempre definita.

Per la preservazione chiamiamo $q, r$ i valori prima del corpo e $q$', $r$' quelli dopo. Il corpo produce $q' = q + 1$ e $r' = r - y$, e quindi

$$q'y + r' = (q+1)y + (r-y) = q\,y + r = x.$$

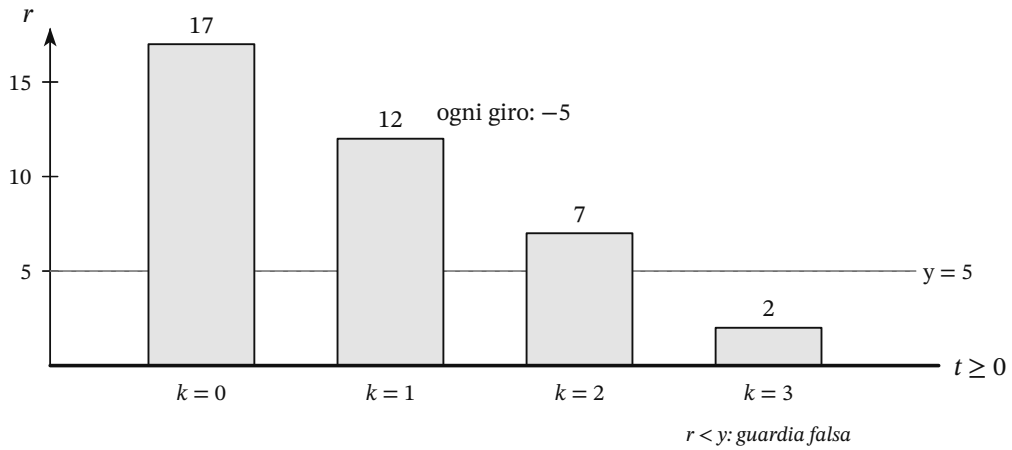
La guardia $r \geq y$ ci assicura che $r$' non sia negativo, e siccome il corpo non assegna mai $y$, anche $y > 0$ rimane vera. Le somme e le sottrazioni sono sempre definite, quindi il corpo termina senza errori e ristabilisce l'invariante.

### 7.3.2 Progresso e risultato

L'invariante ci dà $t = r \geq 0$, cioè il limite inferiore. Se prima del corpo $r = V$, dopo il corpo $r' = V - y$, che è strettamente minore di $V$ proprio perché $y > 0$. All'uscita la guardia è falsa, quindi $r < y$, e insieme all'invariante otteniamo

$$x = q\,y + r \wedge 0 \leq r < y.$$

È esattamente la caratterizzazione del quoziente e del resto della divisione di $x$ per $y$. Il programma termina e li calcola per ogni $x \geq 0$ e $y > 0$. Vale la pena guardare che cosa succederebbe con $y = 0$. La guardia diventerebbe $r \geq 0$, che è sempre vera, la sottrazione lascerebbe $r$ invariato e il ciclo girerebbe per sempre. È l'obbligo di progresso a fallire, e questo spiega concretamente perché l'ipotesi sul divisore è necessaria.




Per $x = 17, y = 5$ il resto scende da 17 a 2. Il limite inferiore e la diminuzione intera spiegano la terminazione; l'invariante spiega il significato del risultato.

## 7.4 Somma dei primi interi: scegliere l'invariante

Nel caso della divisione l'invariante si leggeva quasi direttamente nella traccia. Vediamo ora un esempio in cui conviene costruirlo con un po' più di metodo. Vogliamo sommare gli interi da 0 a $n-1$, con $n \geq 0$ che il programma non modifica. Invece di ricordare tutti gli addendi, teniamo soltanto la somma di quelli già visti e l'indice del prossimo.

```
x := 0; s := 0;
while x < n do
    s := s + x;
    x := x + 1
endw
```

A ogni controllo della guardia, $x$ è il prossimo intero da aggiungere, e quindi gli addendi già sommati sono $0, 1, \ldots, x-1$. Questa lettura del programma suggerisce l'invariante

$$I \equiv s = \sum_{i=0}^{x-1} i \wedge 0 \leq x \leq n.$$

Quando $x = 0$ la somma è vuota e vale zero, quindi l'inizializzazione stabilisce l'invariante. I limiti su $x$ hanno due compiti, perché ci permettono di interpretare la parte già percorsa e di riconoscere quando siamo arrivati in fondo.

### 7.4.1 Preparare il giro successivo

Per verificare la preservazione ragioniamo all'indietro, come abbiamo imparato con le sequenze. Che cosa deve valere prima dell'ultimo comando, `x := x + 1`, perché dopo valga l'invariante? Sostituendo $x+1$ al posto di $x$ otteniamo

$$s = \sum_{i=0}^{x} i \wedge 0 \leq x+1 \leq n.$$

Risalendo ancora attraverso `s := s + x`, prima del corpo deve valere

$$s + x = \sum_{i=0}^{x} i \wedge 0 \leq x + 1 \leq n.$$

La prima parte segue dall'invariante, grazie all'identità

$$\sum_{i=0}^{x} i = \sum_{i=0}^{x-1} i + x.$$

La seconda parte segue dalla guardia, perché $x < n$ e $0 \leq x$, con $x$ e $n$ interi, danno $0 \leq x + 1 \leq n$. Entrambe le richieste sono soddisfatte, e tutte le espressioni coinvolte sono definite.

### 7.4.2 Quanto lavoro rimane?

Per la terminazione scegliamo $t = n - x$, cioè il numero di addendi ancora da aggiungere. Il limite inferiore viene da $x \leq n$, che dà $t \geq 0$. Ogni giro aumenta $x$ di uno e lascia fisso $n$, quindi se $t$ valeva $V$ ora vale $V - 1$. All'uscita abbiamo sia $x \geq n$, dalla guardia falsa, sia $x \leq n$, dall'invariante, e quindi $x = n$, che trasforma l'invariante nella somma richiesta. Se $n = 0$ non si esegue nessun giro e il risultato è correttamente zero.

L'ordine dei due assegnamenti nel corpo è essenziale. Aggiungiamo il vecchio valore di $x$ e solo dopo lo incrementiamo. Se li scambiassimo sommeremmo gli interi da 1 a $n$, che è un problema diverso.

## 7.5 Programmi annotati e proprietà non modificate

Una prova come quella della somma si può scrivere direttamente sul programma, come una serie di annotazioni nei punti importanti. Prima del `while` indichiamo $I$ e $t = n - x$, all'inizio del corpo $I \wedge x < n$, fra i due assegnamenti l'asserzione intermedia appena ricavata e dopo il ciclo $I \wedge x \geq n$. Queste annotazioni non vengono eseguite, ma dichiarano gli obblighi che chi legge deve poter giustificare.

### 7.5.1 Trasportare ciò che il comando non tocca

Capita spesso di dover portare attraverso un comando un'informazione che il comando non modifica. Indichiamo con $\mathrm{change}(C)$ l'insieme dei nomi che compaiono come destinazione di un assegnamento in $C$. Sono i nomi che il comando potrebbe modificare, anche se in una particolare esecuzione un ramo non viene eseguito. Con $\mathrm{free}(R)$ indichiamo invece le variabili libere dell'asserzione $R$. Se

$$\mathrm{free}(R) \cap \mathrm{change}(C) = \varnothing,$$

il comando non tocca nessuna delle variabili di cui $R$ parla, e quindi la verità di $R$ non cambia durante un'esecuzione di $C$ che va a buon fine. Dalla tripla $\{P\}\,C\,\{Q\}$ possiamo allora ricavare $\{R \wedge P\}\,C\,\{R \wedge Q\}$. Nella divisione, per esempio, il corpo assegna soltanto $q$ e $r$, e per questo $y > 0$ passa intatta da un giro all'altro.

### 7.5.2 Combinare due garanzie

Due prove dello stesso comando si possono anche combinare. Da $\{P_1\}\,C\,\{Q_1\}$ e $\{P_2\}\,C\,\{Q_2\}$ ricaviamo $\{P_1 \wedge P_2\}\,C\,\{Q_1 \wedge Q_2\}$, perché uno stato che soddisfa entrambi gli ingressi gode di entrambe le garanzie sul risultato. Nel nostro linguaggio deterministico c'è una sola esecuzione, e quindi le due conclusioni valgono nello stesso stato finale.

Allo stesso modo, da $\{P\}\,C\,\{Q\}$ e $\{R\}\,C\,\{Q\}$ ricaviamo $\{P \vee R\}\,C\,\{Q\}$. Basta distinguere quale dei due ingressi vale e applicare la prova corrispondente.

## 7.6 Per proseguire: eliminare gli incroci

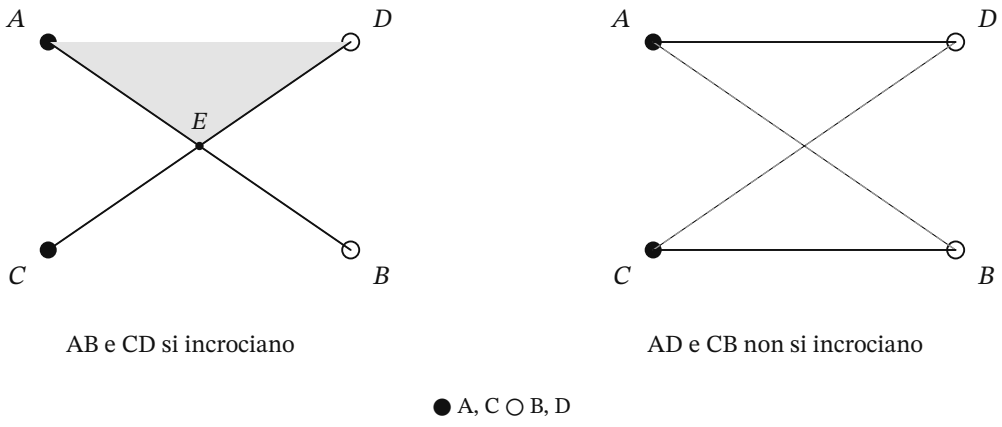
Un esempio geometrico mostra che la funzione di terminazione va cercata pensando al problema, e non soltanto guardando il codice. Sul piano ci sono $N$ punti blu e $N$ punti rossi, tutti distinti e senza tre punti allineati. Vogliamo abbinare ogni punto blu a un punto rosso con dei segmenti, in modo che nessuna coppia di segmenti si incroci. Il procedimento è semplice. Partiamo da un abbinamento qualsiasi, e finché due segmenti si incrociano li sostituiamo con i due collegamenti che non si incrociano e rispettano i colori.

L'invariante dice che ogni punto appartiene esattamente a un segmento e che ogni segmento unisce colori diversi. Lo scambio lo mantiene, perché ricollega gli stessi quattro estremi, ciascuno una sola volta. Quando il procedimento si ferma non ci sono più incroci, quindi l'obiettivo è raggiunto. Resta da capire perché il procedimento debba fermarsi.

Contare gli incroci sembra la scelta naturale, ma non funziona, perché eliminare l'incrocio scelto può crearne di nuovi con altri segmenti. Proviamo allora con la somma delle lunghezze di tutti i segmenti. Supponiamo che $AB$ e $CD$ si incontrino nel punto $E$ e che i colori permettano i nuovi segmenti $AD$ e $CB$. La disuguaglianza triangolare dà

$$\ell(AD) < \ell(AE) + \ell(ED), \quad \ell(CB) < \ell(CE) + \ell(EB).$$

Le disuguaglianze sono strette perché i triangoli coinvolti non sono degeneri, grazie all'ipotesi che non ci siano tre punti allineati. Sommandole troviamo che la nuova lunghezza totale è strettamente minore della vecchia, dato che $\ell(AE) + \ell(EB) = \ell(AB)$ e $\ell(CE) + \ell(ED) = \ell(CD)$, mentre tutti gli altri segmenti restano come erano.




I percorsi spezzati che passano per $E$ sono più lunghi dei nuovi collegamenti rettilinei. Il risparmio riguarda la lunghezza totale.

C'è però un ostacolo. Le lunghezze sono numeri reali, e abbiamo visto che una quantità reale positiva può diminuire per sempre. Qui ci salva il fatto che i punti sono fissati e gli abbinamenti possibili sono finiti, esattamente $N!$, perché a ogni punto blu associamo un punto rosso diverso. Anche i valori che la lunghezza totale può assumere sono quindi in numero finito. Una diminuzione stretta impedisce di tornare a un abbinamento già visto, e perciò non può continuare all'infinito. Abbiamo ottenuto una prova completa di terminazione, basata sulla finitezza degli stati possibili invece che su un numero naturale.

---

### Verifica 7

**1.** Per `while i < n do i := i + 1 endw`, con $I \equiv 0 \le i \le n$ e $n$ fisso, proponi una funzione di terminazione e verifica limite inferiore, progresso e uscita.

**2.** Nel programma della somma scambia i due assegnamenti del corpo. Trova un invariante che descriva la somma ora calcolata e spiega il risultato finale, compreso il caso $n = 0$.

# Costruire un ciclo: il significato delle variabili

**In breve**
Finora abbiamo ricevuto i cicli già scritti e ci siamo limitati a verificarli. Il passo più interessante, però, è inventarli. Partendo dal risultato che vogliamo e da qualche esempio piccolo, decideremo prima che cosa deve ricordare ciascuna variabile durante il calcolo, e solo dopo scriveremo i comandi che mantengono quel significato. Il metodo funzionerà per il fattoriale, il massimo comun divisore, la radice cubica e la successione di Fibonacci.

## 8.1 Dalla postcondizione a una proprietà da mantenere

Per trovare il quoziente della divisione di 17 per 5 possiamo togliere 5 più volte e contare quante volte lo facciamo. Dopo due sottrazioni abbiamo tolto 10 e ci resta 7, e l'uguaglianza $17 = 2 \cdot 5 + 7$ descrive esattamente il punto a cui siamo arrivati. Il resto non è ancora abbastanza piccolo, ma l'uguaglianza è già vera.

Questa osservazione spiega la scelta fatta nei capitoli precedenti. La postcondizione $x = q\,y + r \wedge 0 \le r < y$ è stata divisa in due parti, una relazione da mantenere, $x = q\,y + r \wedge r \ge 0$, e una condizione da raggiungere, $r < y$. La prima, insieme al vincolo $y > 0$, diventa l'invariante. La seconda suggerisce la guardia, che è la sua negazione, `r >= y`. In più, la variabile $r$ rende visibile il testimone dell'esistenziale che definisce il quoziente, cioè il resto che rende vera $x = q\,y + r$ con $0 \le r < y$.

In generale cerchiamo due proprietà $I$ e $G$ tali che $I \wedge G$ implichi la postcondizione. Vogliamo mantenere vera $I$ e continuare a girare finché $G$ è falsa. Servono poi un'inizializzazione che renda vera $I$, un corpo che la ristabilisca e una prova di terminazione. Un vincolo pratico limita le scelte, perché la guardia deve essere un'espressione del linguaggio, e non ogni scomposizione matematica della postcondizione si lascia tradurre in un programma.

**Concetto chiave · Invariante come significato delle variabili**
Un buon invariante spiega che cosa rappresentano i valori presenti durante il calcolo. Si rafforza con i vincoli necessari per leggere le espressioni, eseguire il passo successivo e dimostrare la terminazione.

## 8.2 Fattoriale: ricordare il prodotto già calcolato

Il fattoriale $n!$ è il prodotto degli interi da 1 a $n$, e per convenzione $0! = 1$. Supponiamo che $n \ge 0$ sia un intero che il programma non modifica. Per $n = 4$ vogliamo ottenere uno dopo l'altro i prodotti $1$, $1 \cdot 1$, $1 \cdot 1 \cdot 2$, $1 \cdot 1 \cdot 2 \cdot 3$ e $1 \cdot 1 \cdot 2 \cdot 3 \cdot 4$. Ci servono quindi una variabile per il prodotto e una per il prossimo fattore da usare.

```
fatt := 1; x := 1;
while x <= n do
    fatt := fatt * x;
    x := x + 1
endw
```

Quando stiamo per usare il fattore $x$, abbiamo già moltiplicato tutti gli interi fino a $x-1$. Questa frase, scritta in formule, è l'invariante che cerchiamo:

$$I \equiv \text{fatt}{=}(x-1)! \wedge 1 \le x \le n+1.$$

I due limiti hanno ciascuno un motivo. Quello inferiore, 1, garantisce che il fattoriale scritto nell'asserzione abbia un argomento naturale. Quello superiore, $n+1$, comprende lo stato finale, perché dopo aver usato il fattore $n$ il contatore avanza ancora una volta.

### 8.2.1 Controllare il significato del prodotto

All'inizio $x=1$ e fatt$=1=0!$, e inoltre $1 \le n+1$, quindi l'invariante vale. Supponiamo ora che valga e che la guardia $x \le n$ sia vera. Il primo assegnamento produce fatt' $=(x-1)! \cdot x = x!$, e il secondo porta $x$ al nuovo valore $x' = x+1$. La relazione che vogliamo ritrovare è fatt' $=(x'-1)!$, cioè proprio fatt' $=x!$, che abbiamo appena ottenuto. Anche i limiti restano veri, perché da $x \le n$ segue $1 \le x' \le n+1$.

### 8.2.2 Fermarsi dopo l'ultimo fattore

Come funzione di terminazione possiamo prendere $n+1-x$, che sotto l'invariante è un intero non negativo e diminuisce di 1 a ogni giro. Tutte le espressioni del corpo e della guardia sono definite, quindi il ciclo termina. All'uscita la guardia è falsa e $x > n$, e insieme a $x \le n+1$, con $x$ e $n$ interi, questo dà $x = n+1$. L'invariante diventa allora fatt$=n!$.

Se $n=0$ non si entra mai nel ciclo e il risultato rimane 1, come deve essere. Questo caso spiega anche perché non abbiamo scelto la funzione $n-x$, che sembrerebbe più naturale. Nello stato finale varrebbe $-1$, e la regola del corso chiede che la funzione di terminazione sia non negativa in tutti gli stati dell'invariante, compreso l'ultimo.

## 8.3 MCD: trasformare i dati senza cambiare la risposta

Il massimo comune divisore di 18 e 12 coincide con quello di 6 e 12. Il motivo è che un intero divide sia 18 sia 12 esattamente quando divide sia $18-12$ sia 12. In un verso basta guardare la differenza, nell'altro la somma $18 = (18-12)+12$. Sottrarre il numero minore dal maggiore non cambia quindi l'insieme dei divisori comuni, e in particolare non cambia il più grande.

Questa idea suggerisce un algoritmo. Con $a, b > 0$ interi che il programma non modifica, lavoriamo su due copie e continuiamo a sottrarre finché diventano uguali.

```
x := a; y := b;
while x != y do
    if x > y
        then x := x - y
        else y := y - x
    fi
endw
```

Qui l'invariante non descrive un risultato accumulato poco a poco, come nel fattoriale. Dice invece che la risposta è rimasta la stessa mentre i dati cambiavano:

$$I \equiv x > 0 \wedge y > 0 \wedge \text{mcd}(x, y) = \text{mcd}(a, b).$$

L'inizializzazione rende vera questa proprietà. Se $x > y$, allora $x - y > 0$, e per quanto abbiamo appena visto il MCD non cambia. Se invece si esegue l'altro ramo, sappiamo sia che $x \neq y$, dalla guardia del ciclo, sia che $x > y$ è falsa, dalla guardia del condizionale. Insieme danno $x < y$, e vale il ragionamento simmetrico. La guardia del ciclo serve quindi anche a giustificare il secondo ramo.

### 8.3.1 Una somma che diminuisce

Per la terminazione non serve sapere quanti passi saranno necessari, ma basta trovare una quantità che scenda. La somma $x+y$ è positiva sotto l'invariante. Nel primo ramo il suo nuovo valore è $(x-y)+y=x$, strettamente minore di $x+y$ perché $y>0$, e nel secondo ramo il nuovo valore è $y$, che per lo stesso motivo è minore. Quindi il ciclo termina.

All'uscita $x=y>0$, e il MCD di due numeri positivi uguali è quel numero stesso. L'invariante ci dà allora $x=\mathrm{mcd}(a,b)$. Partendo per esempio da $(x,y)=(12,18)$, passiamo a $(12,6)$ e poi a $(6,6)$, e la risposta è 6.

## 8.4 Radice cubica: costruire potenze con differenze

Dato un intero $x\geq 0$, cerchiamo l'intero $r\geq 0$ per cui $r^3\leq x<(r+1)^3$, cioè la parte intera della radice cubica di $x$. Con $x=10$ i primi cubi sono $0,1,8,27$, e la risposta è 2, perché $8\leq 10<27$.

L'idea più semplice è provare i cubi in ordine fino al primo che supera $x$. Per passare da un cubo al successivo senza moltiplicazioni possiamo usare l'identità

$$(b+1)^3=b^3+3b^2+3b+1.$$

Il cubo da solo non basta, perché la formula chiede anche il quadrato. Teniamo allora l'indice in $b$, il quadrato in $a$ e il cubo in $y$, e per aggiornare il quadrato usiamo a sua volta $(b+1)^2=b^2+2b+1$.

```
a := 0; b := 0; y := 0;
while y <= x do
    y := y + 3*a + 3*b + 1;
    a := a + 2*b + 1;
    b := b + 1
endw
```

L'ordine dei comandi è studiato con cura. Il primo assegnamento deve leggere i vecchi `a` e `b`, il secondo il vecchio `b`, e per questo aumentiamo `b` soltanto alla fine.

### 8.4.1 Seguire il calcolo per 10

La tabella mostra i valori ai controlli della guardia.

| $b$ | $a=b^2$ | $y=b^3$ | $y\leq 10$ |
|---|---|---|---|
| 0 | 0 | 0 | vero |
| 1 | 1 | 1 | vero |
| 2 | 4 | 8 | vero |
| 3 | 9 | 27 | falso |

Guardiamo da vicino l'ultimo giro, che parte da $a=4$, $b=2$ e $y=8$. Il primo comando calcola $8+3\cdot 4+3\cdot 2+1=27$, il secondo $4+2\cdot 2+1=9$ e il terzo porta $b$ a 3. Al controllo successivo la guardia è falsa e usciamo.

A questo punto $b$ ha superato la risposta di uno. Il cubo attuale, 27, supera 10, mentre il precedente, $2^3=8$, era ancora ammesso. Questo è il fatto che l'invariante dovrà ricordare.

### 8.4.2 Ricordare anche il cubo precedente

Ai controlli della guardia vogliamo che valga

$$I\equiv a=b^2\wedge y=b^3\wedge (b-1)^3\leq x\wedge b\geq 0\wedge x\geq 0.$$

All'inizio $a = b = y = 0$, e la condizione sul cubo precedente vale perché $(b-1)^3 = -1 \le x$. In questo stato non c'è nessun cubo già provato dal programma, e la condizione serve soltanto a far funzionare la stessa formula anche all'inizio.

Supponiamo ora vera la guardia. Le due identità sulle potenze assicurano che i nuovi valori di $a$ e $y$ siano il quadrato e il cubo del nuovo indice $b' = b+1$. Resta da controllare il cubo precedente. Prima del giro la guardia ci dice che $b^3 = y \le x$, e dopo il giro il cubo precedente è proprio $(b'-1)^3 = b^3$, quindi anche questa parte dell'invariante rimane vera. I limiti $b' \ge 0$ e $x \ge 0$ sono evidenti.

### 8.4.3 Come trovare una quantità che diminuisce

Il cubo $y$ cresce, e verrebbe spontaneo misurare il lavoro che resta con la distanza $x - y$. Nell'ultimo stato, però, questa quantità è negativa, e per $x = 10$ vale $10 - 27 = -17$, quindi non rispetta la non negatività richiesta.

L'invariante ci suggerisce una quantità migliore. Il cubo *precedente* non supera mai $x$, quindi la distanza $x - (b-1)^3$ è sempre non negativa. Aggiungendo 1 scegliamo

$$t = x + 1 - (b-1)^3,$$

che vale almeno 1 in ogni stato dell'invariante, compreso quello finale. Possiamo provarla sulle stesse quattro righe di prima, con $x = 10$.

| $b$ | $(b-1)^3$ | $t = 11 - (b-1)^3$ |
|---|---|---|
| 0 | $-1$ | 12 |
| 1 | 0 | 11 |
| 2 | 1 | 10 |
| 3 | 8 | 3 |

La traccia suggerisce che $t$ diminuisca, e ora lo dimostriamo per ogni giro. Dopo il corpo $b' = b+1$, quindi $t' = x + 1 - b^3$, e sottraendo troviamo

$$t - t' = b^3 - (b-1)^3 = 3b(b-1) + 1.$$

Per $b = 0$ e per $b = 1$ la differenza vale 1, e per ogni intero $b \ge 2$ il prodotto $b(b-1)$ è positivo. In tutti i casi ammessi $t - t' > 0$, cioè $t$ scende strettamente. Siccome la guardia e il corpo usano soltanto operazioni sempre definite sugli interi, ogni giro termina senza errori, e la funzione $t$ esclude che i giri siano infiniti.

### 8.4.4 Leggere il risultato all'uscita

All'uscita la guardia è falsa, cioè $y > x$, e l'invariante ci dà

$$(b-1)^3 \le x < b^3.$$

All'inizio la guardia era vera, perché $0 \le x$, quindi il corpo è stato eseguito almeno una volta e all'uscita $b \ge 1$. La risposta $b - 1$ è quindi non negativa, ed è esattamente la parte intera della radice cubica.

**Attenzione · Quale variabile contiene la radice?**
Il ciclo lascia in `b` un valore in più: per $x = 0$ lascia 1, per $x = 8$ lascia 3. Per consegnare la risposta in `b`, aggiungiamo dopo il ciclo `b := b - 1`. L'invariante riguarda il ciclo prima di questa correzione.

## 8.5 Fibonacci: tenere il valore che manca

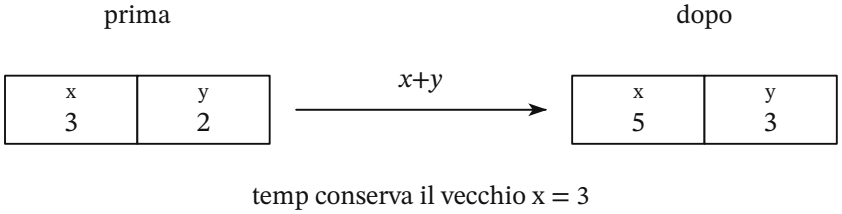
La successione di Fibonacci parte da $\text{fib}(0) = 0$ e $\text{fib}(1) = 1$, e ogni termine successivo è la somma dei due precedenti. Per calcolare $\text{fib}(k)$ con $k \geq 1$ vorremmo mantenere $x = \text{fib}(i)$ e far avanzare $i$ fino a $k$.

Il valore successivo sarebbe $\text{fib}(i+1) = x + \text{fib}(i-1)$. La formula spiega il passo ma non è ancora un comando, perché `fib` non è un'operazione del nostro linguaggio. Teniamo allora il valore che manca in una seconda variabile $y$, e l'invariante diventa

$$I \equiv x = \text{fib}(i) \wedge y = \text{fib}(i-1) \wedge 1 \leq i \leq k.$$

Per avanzare di un passo, il nuovo $x$ deve valere il vecchio $x + y$ e il nuovo $y$ deve valere il vecchio $x$. Se per esempio $x = 3$ e $y = 2$, i nuovi valori devono essere 5 e 3. La sequenza `x := x+y; y := x` produrrebbe invece 5 e 5, perché il secondo comando legge il nuovo $x$. È lo stesso problema dello scambio, e lo risolviamo allo stesso modo, mettendo da parte il vecchio $x$.

```
x := 1; i := 1; y := 0;
while i != k do
    temp := x;
    x := x + y;
    y := temp;
    i := i + 1
endw
```




La coppia dei termini consecutivi si sposta di un posto. La variabile temporanea permette di usare il vecchio $x$ anche dopo averlo aggiornato.

### 8.5.1 Controllare il passo e l'uscita

L'inizializzazione usa $\text{fib}(1) = 1$ e $\text{fib}(0) = 0$, quindi l'invariante vale. Dentro il corpo sappiamo che $1 \leq i \leq k$ e che $i \neq k$, e quindi $i < k$. Indichiamo con un apice i valori dopo il corpo. La somma produce il termine successivo e la variabile d'appoggio porta il termine che deve diventare il precedente,

$$x' = \text{fib}(i) + \text{fib}(i-1) = \text{fib}(i+1),$$

$$y' = \text{fib}(i), \qquad i' = i + 1.$$

Riscritte rispetto al nuovo indice, queste uguaglianze dicono $x' = \text{fib}(i')$ e $y' = \text{fib}(i'-1)$, e da $i < k$ segue $1 \leq i' \leq k$. L'invariante è ristabilito.

La quantità $k - i$ è non negativa e diminuisce di 1 a ogni giro, e tutte le operazioni sono definite, quindi il ciclo termina. All'uscita $i = k$ e l'invariante dà $x = \text{fib}(k)$. Se $k = 1$ la guardia è subito falsa e il risultato è già pronto. Il caso $k = 0$ è escluso da questa specifica, e la verifica alla fine del capitolo chiede come includerlo.

## 8.6 Per proseguire: seguire una ricorrenza che dimezza l'indice

Un altro problema del corso mostra che far crescere un contatore non è sempre la strada migliore. Definiamo $f(0)=0$ e $f(1)=1$, e per $j\geq 2$ poniamo $f(j)=f(j/2)$ se $j$ è pari e $f(j)=f((j-1)/2)+f((j+1)/2)$ se è dispari. Gli argomenti a destra sono sempre più piccoli di $j$, quindi la definizione permette di calcolare ogni valore a partire dai due casi iniziali.

Calcoliamo per esempio $f(19)$, scrivendo a ogni passo il risultato come combinazione di due valori con indici consecutivi:

$$f(19)=f(9)+f(10)=f(4)+2f(5)=3f(2)+2f(3)=5f(1)+2f(2)=7.$$

Il fatto che compaiano sempre due indici consecutivi suggerisce l'invariante

$$I\equiv 0\leq x\leq n\wedge f(n)=kf(x)+hf(x+1),$$

dove $n\geq 0$ non viene modificato. Con $x=n$, $k=1$ e $h=0$ l'uguaglianza è vera. Quando $x=0$ diventa $f(n)=k\cdot 0+h\cdot 1=h$, e così sappiamo già quale guardia scegliere e dove troveremo il risultato.

Ricaviamo ora il corpo, scrivendo $q=x\operatorname{div}2$. Se $x=2q$ con $q>0$, allora $f(x)=f(q)$ e $f(x+1)=f(q)+f(q+1)$, e quindi

$$kf(x)+hf(x+1)=(k+h)f(q)+hf(q+1).$$

Basta allora sommare $h$ a $k$ e dimezzare $x$. Se invece $x=2q+1$, le stesse identità danno

$$kf(x)+hf(x+1)=kf(q)+(k+h)f(q+1),$$

e questa volta basta sommare $k$ a $h$ e dimezzare $x$. Per $x=1$ la prima identità del caso dispari vale ancora, perché $f(1)=f(0)+f(1)$. Possiamo ora scrivere il programma, usando il test `x mod 2 = 0` per riconoscere gli indici pari.

```
x := n; k := 1; h := 0;
while x != 0 do
    if x mod 2 = 0
        then k := h + k
        else h := h + k
    fi;
    x := x div 2
endw
```

Le uguaglianze appena ottenute dimostrano che l'invariante viene ristabilito. Sotto la guardia vale $x>0$, e quindi $0\leq x\operatorname{div}2<x$, il che fa di $x$ stessa una funzione di terminazione. Il divisore 2 non è mai nullo. All'uscita $h=f(n)$, anche quando $n=0$ e il corpo non viene mai eseguito. Le asserzioni hanno guidato tutta la costruzione, e il programma che ne è uscito usa soltanto somme, divisione intera e modulo.

---

**Verifica 8**

**1.** Vogliamo mantenere vera $x\,y+p\,q=n$ mediante `x := x-p; q := E`. Trova un'espressione $E$ che funzioni per tutti gli interi e giustifica anche il caso $p=0$.

**2.** Nel programma di Fibonacci modifica soltanto l'inizializzazione e l'invariante per calcolare anche il caso $k=0$. Puoi mantenere la stessa forma del corpo usando due termini consecutivi a indici non negativi? Indica quale variabile conterrà il risultato.

# Array: indici, aggiornamenti e zone

**In breve**
Quando i dati sono molti, come i voti di una classe, non possiamo dare un nome a ciascuno, e li raccogliamo in un array dove ogni valore ha la sua posizione. Il rischio nuovo è sbagliare posizione, leggendo un indice che non esiste o perdendo un valore durante uno spostamento. In questo capitolo impariamo a controllare gli indici, a ragionare sugli aggiornamenti e soprattutto a pensare un ciclo come un confine che si sposta, separando la parte già sistemata da quella che resta da esaminare.

Per sommare i voti di un elenco vorremmo tenere molti valori sotto un solo nome. Un **array** fa esattamente questo, raccogliendo una sequenza di valori, e un **indice** permette di scegliere una posizione al suo interno. La comodità ha però un prezzo, perché dovremo controllare ogni volta quali posizioni possiamo leggere e modificare.

## 9.1 Array come funzioni sugli indici

La dichiarazione `v : array [0,4) of int` descrive quattro interi con indici $0, 1, 2, 3$. L'estremo destro, 4, segna la fine dell'array e non è una posizione che si possa leggere. In generale un array omogeneo, cioè con elementi tutti dello stesso tipo, è una funzione

$$v\colon [a, b) \to A,$$

dove $a \le b$ sono naturali e $A$ è $\mathbb{Z}$ oppure Bool. L'intervallo $[a, b)$ comprende gli indici $i$ con $a \le i < b$, quindi ha $b - a$ posizioni, ed è vuoto quando $a = b$. Per il dominio degli indici validi scriviamo $\mathrm{dom}(v) = [a, b)$.

Con gli array, lo stato associa al nome $v$ un'intera funzione $\sigma(v)$. Se $\sigma(v) = \{0 \mapsto 2, 1 \mapsto 1, 2 \mapsto 10, 3 \mapsto 6\}$, leggere `v[2]` dà 10. Per leggere un elemento si calcola prima l'espressione dell'indice e poi si applica la funzione,

$$\mathcal{E}(v[E], \sigma) = \sigma(v)(\mathcal{E}(E, \sigma)),$$

e la formula ha senso soltanto se il valore dell'indice esiste e cade nel dominio.

Un indice può contenere a sua volta altre letture. Per calcolare `v[v[0]+v[1]]` leggiamo prima 2 e 1, li sommiamo ottenendo 3 e solo allora leggiamo `v[3]`, che vale 6. Gli indici da controllare sono quindi tre, i due interni, 0 e 1, e quello esterno, 3. Se la somma fosse stata 4, le due letture interne sarebbero state lecite ma quella esterna sarebbe fallita.

**Concetto chiave · Definitezza di un accesso**
Per un'espressione indice intera $E$,

$$\mathrm{def}(v[E]) \equiv \mathrm{def}(E) \wedge E \in \mathrm{dom}(v).$$

La correttezza richiede che ogni lettura effettivamente eseguita abbia un indice valido. Per un array su $[0, n)$, il solo vincolo $x \le n$ non autorizza a leggere `v[x]`: occorre $0 \le x < n$.

Le possibili cause di indefinitezza diventano quindi due, i divisori nulli e gli indici fuori dal dominio. I tipi restano un controllo a parte. Un array di booleani produce booleani, che non si possono usare come indici interi. Inoltre gli operatori del linguaggio lavorano sui singoli elementi, e una scrittura come `v+w`, intesa come somma di due array interi, non è ammessa.

## 9.2 Aggiornamento selettivo: prima valutare, poi modificare

Il comando `v[E] := E'` cambia una sola posizione dell'array. Come per l'assegnamento semplice, sia l'indice sia il valore a destra si calcolano nello stato precedente. Con l'array di prima, il comando `v[v[0]] := v[1]+5` calcola l'indice 2 e il valore 6, e soltanto dopo cambia `v[2]`, ottenendo $\{0 \mapsto 2, 1 \mapsto 1, 2 \mapsto 6, 3 \mapsto 6\}$.

Per descrivere l'effetto in modo preciso chiamiamo $j = \mathcal{E}(E, \sigma)$ e $d = \mathcal{E}(E', \sigma)$. Se entrambi sono definiti e $j \in \mathrm{dom}(v)$, costruiamo la nuova sequenza $w = \sigma(v)[d/j]$, che vale $d$ all'indice $j$ e coincide con $\sigma(v)$ in tutti gli altri. Poi aggiorniamo lo stato,

$$\mathcal{C}(v[E] := E', \sigma) = \sigma[w/v].$$

Gli aggiornamenti sono quindi due, uno dentro l'altro. Quello interno cambia una posizione della sequenza, quello esterno cambia il valore associato al nome $v$. Il dominio degli indici resta lo stesso.

### 9.2.1 La condizione prima di un aggiornamento

Per verificare una postcondizione $P$ ragioniamo come con l'assegnamento semplice, sostituendo però l'intera sequenza. Scriviamo simbolicamente $w = v[E'/E]$ per la sequenza aggiornata, e l'assioma richiede prima del comando

$$\mathrm{def}(E) \land \mathrm{def}(E') \land E \in \mathrm{dom}(v) \land P[w/v].$$

Dopo `v[E] := E'` vale allora $P$, e come sempre la regola di conseguenza permette di partire da una precondizione più forte. Leggere $P[w/v]$ richiede di distinguere due casi. In posizione $i$ la nuova sequenza vale $E'$ se $i = E$, e il vecchio $v[i]$ se $i \neq E$.

**Esempio guidato · Estendere un prefisso di somme**

Supponiamo $1 \le k < n$ e che, prima della posizione $k$, l'array contenga già le somme $a[i] = \sum_{j=0}^{i} j$ per ogni $0 \le i < k$. Eseguiamo `a[k] := a[k-1]+k`.

I vincoli su $k$ rendono validi sia l'indice $k$ sia l'indice $k-1$. Per dimostrare che la proprietà vale ora fino a $k$ compreso, separiamo i due casi. Se $i < k$, la posizione $i$ non viene toccata e contiene ancora la somma corretta. Se $i = k$, il nuovo valore è

$$a[k-1] + k = \sum_{j=0}^{k-1} j + k = \sum_{j=0}^{k} j.$$

Abbiamo così controllato tutti gli indici $0 \le i \le k$. È la distinzione fra la posizione modificata e quelle rimaste uguali a rendere possibile la prova di una postcondizione quantificata.

## 9.3 Incrementare un array: un confine tra fatto e da fare

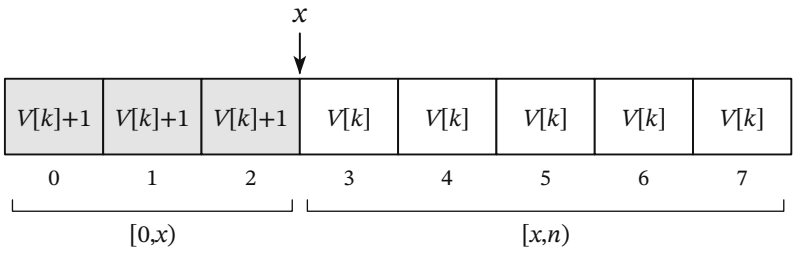
Vogliamo aggiungere 1 a ogni elemento di `a : array [0,n) of int`, con $n \ge 0$. Chiamiamo $V$ la sequenza iniziale, una variabile di specifica che il programma non modifica, e il risultato richiesto diventa $\forall k \in [0, n).\, a[k] = V[k] + 1$.

L'idea che guida il ciclo è un confine $x$ che attraversa l'array. A sinistra del confine gli elementi sono già stati incrementati, da $x$ in poi sono ancora quelli iniziali. All'inizio il confine sta in 0, e alla fine vogliamo che arrivi a $n$. Tradotta in formule, questa immagine dà l'invariante

$$I \equiv 0 \le x \le n \land (\forall k \in [0, x).\, a[k] = V[k] + 1)$$

$$\land (\forall k \in [x, n).\, a[k] = V[k]).$$

La seconda riga potrebbe sembrare superflua, ma senza di essa non potremmo concludere nulla. Per sapere che incrementando `a[x]` otteniamo $V[x]+1$, dobbiamo sapere che prima del comando `a[x]` vale ancora $V[x]$.




Il confine $x$ separa le posizioni già incrementate da quelle ancora uguali alla sequenza iniziale $V$. Il confine può anche essere 0 oppure $n$.

```
x := 0;
while x < n do
    a[x] := a[x] + 1;
    x := x + 1
endw
```

La precondizione è $n \geq 0 \wedge a = V$, dove l'uguaglianza vuol dire che i due array coincidono elemento per elemento. Con $x = 0$ la parte già elaborata è vuota, e la proprietà sul resto coincide con la precondizione, quindi l'invariante vale.

Prima del corpo, l'invariante e la guardia danno $0 \leq x < n$, e così la lettura e la scrittura di `a[x]` sono lecite. Dopo il primo comando le posizioni prima di $x$ sono ancora incrementate, quella in $x$ passa da $V[x]$ a $V[x]+1$ e quelle successive sono ancora quelle iniziali. È la stessa divisione di prima, con il confine spostato in $x+1$, e il secondo comando dà a $x$ proprio quel valore, ristabilendo l'invariante.

### 9.3.1 Quando il confine raggiunge la fine

La quantità $n - x$ è non negativa e diminuisce di 1 a ogni giro, quindi il ciclo termina. Quando la guardia è falsa vale $x \geq n$, e insieme a $x \leq n$ otteniamo $x = n$. A quel punto l'intero array appartiene alla parte elaborata. Se $n = 0$ le proprietà quantificate parlano dell'insieme vuoto e sono vere senza che il corpo venga mai eseguito.

## 9.4 Somma di un array: sostituire la fine con un confine mobile

Questa volta l'array non deve cambiare, e vogliamo soltanto calcolare

$$s = \sum_{i=0}^{n-1} a[i].$$

Un trucco utile per trovare l'invariante è sostituire nella postcondizione l'estremo finale $n$ con un indice di lavoro $x$. Così $s$ diventa la somma delle prime $x$ posizioni. Per $x = 0$ è la somma vuota, che vale 0, e per $x = n$ è esattamente la somma che cerchiamo.

```
x := 0; s := 0;
while x != n do
    s := s + a[x];
    x := x + 1
endw
```

L'invariante è quindi $I \equiv s = \sum_{i=0}^{x-1} a[i] \wedge 0 \le x \le n$. L'uguaglianza spiega che cosa contiene l'accumulatore, mentre i limiti sul contatore servono alla verifica. Sotto la guardia $x \neq n$ danno infatti $0 \le x < n$, e quindi `a[x]` è definito.

All'inizio l'invariante vale per la convenzione sulla somma vuota. Se vale prima del corpo, dopo l'aggiunta otteniamo

$$s' = \sum_{i=0}^{x-1} a[i] + a[x] = \sum_{i=0}^{x} a[i].$$

Poiché il nuovo contatore è $x' = x + 1$, questa è la somma fino a $x' - 1$, e da $x < n$ segue $x' \le n$, quindi l'invariante è ristabilito. Ancora una volta $n - x$ dimostra la terminazione, e la guardia falsa dà direttamente $x = n$. Con $n = 0$ il risultato è 0.

Sostituire una costante della specifica con una variabile ha trasformato la descrizione del risultato completo in una proprietà dei risultati parziali. I limiti aggiunti su $x$ servono a tre cose insieme, cioè a controllare gli indici, a mantenere l'invariante e a dimostrare la terminazione.

## 9.5 Bandiera a due colori: classificare senza perdere elementi

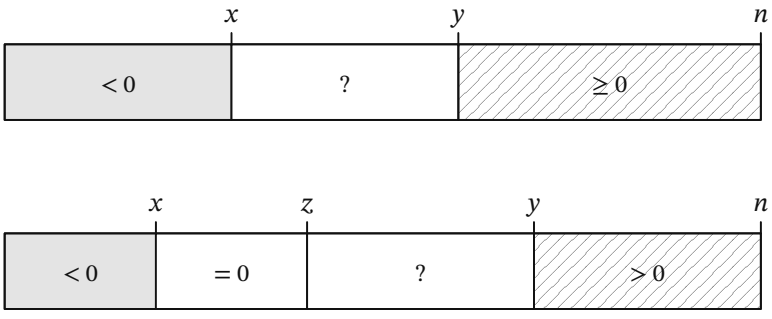
Vogliamo ora riordinare un array in modo che vengano prima tutti i negativi e poi tutti i valori positivi o nulli. Non chiediamo che ciascun gruppo sia ordinato al suo interno, ma chiediamo che nessun valore vada perso, comprese le ripetizioni. Per dirlo scriviamo $\mathrm{perm}(a, A)$ quando $a$ contiene gli stessi elementi della sequenza iniziale $A$, ciascuno con la stessa molteplicità.

Un solo confine non basta più, perché dobbiamo distinguere tre parti, quella già messa a sinistra, quella già messa a destra e quella che non abbiamo ancora guardato. Usiamo allora due confini $x \le y$. Il prefisso $[0, x)$ contiene negativi, il suffisso $[y, n)$ contiene non negativi, e la zona $[x, y)$ è ancora sconosciuta. L'invariante è

$$I \equiv \mathrm{perm}(a, A) \wedge 0 \le x \le y \le n$$

$$\wedge (\forall i \in [0, x).\, a[i] < 0) \wedge (\forall i \in [y, n).\, a[i] \ge 0).$$

Sulla zona sconosciuta l'invariante non dice nulla, perché è esattamente il lavoro che resta da fare. La sua lunghezza $y - x$ suggerisce nello stesso tempo la guardia, `x != y`, e la funzione di terminazione. All'inizio $x = 0$ e $y = n$, e le due zone classificate sono vuote.




In alto, due categorie e una zona ancora sconosciuta. In basso, lo stesso ragionamento è esteso a negativi, zeri e positivi: cambiano i confini, mentre resta centrale la zona da esaminare.

Sotto la guardia vale $x < y$, quindi possiamo leggere sia `a[x]` sia, se serve, `a[y-1]`, perché entrambi gli indici stanno fra 0 e $n - 1$. Guardiamo il primo elemento sconosciuto, `a[x]`. Se è negativo appartiene al prefisso, e basta spostare in avanti il confine $x$. Se invece è non negativo dobbiamo portarlo a destra, nella posizione $y - 1$, e arretrare il confine $y$.

Una semplice copia `a[y-1] := a[x]` cancellerebbe però l'elemento che stava in $y-1$, violando la permutazione. Facciamo quindi uno scambio con una variabile d'appoggio. Dopo lo scambio, in posizione $x$ arriva un valore che non abbiamo ancora esaminato, e quindi in questo ramo $x$ deve restare fermo.

```
x := 0; y := n;
while x != y do
    if a[x] < 0
        then x := x + 1
        else temp := a[y-1];
             a[y-1] := a[x];
             a[x] := temp;
             y := y - 1
    fi
endw
```

### 9.5.1 Perché x resta fermo dopo lo scambio

Seguiamo il programma su quattro elementi, $a = [4, -2, 0, 7]$, partendo da $x = 0$ e $y = 4$. Il primo elemento è non negativo, quindi lo scambiamo con l'ultimo e arretriamo $y$.

| Controllo | $a$ | $x$ | $y$ |
|---|---|---|---|
| iniziale | $[4, -2, 0, 7]$ | 0 | 4 |
| dopo il primo giro | $[7, -2, 0, 4]$ | 0 | 3 |
| dopo il secondo giro | $[0, -2, 7, 4]$ | 0 | 2 |

Dopo il primo giro in posizione 0 è arrivato il 7, che nessuno aveva ancora esaminato. Il secondo giro lo controlla e lo sposta a sua volta a destra, e al suo posto arriva lo 0, che verrà esaminato al giro successivo.

Se dopo il primo scambio avessimo fatto avanzare $x$, avremmo messo il 7 nel prefisso dei negativi, e l'invariante sarebbe crollato subito. Per questo lo scambio sposta soltanto il confine destro.

### 9.5.2 Mantenere le zone e arrivare all'uscita

Controlliamo che entrambi i rami ristabiliscano l'invariante. Nel primo l'array non cambia, e l'unico elemento aggiunto al prefisso è negativo. Nel secondo lo scambio non tocca né il vecchio prefisso $[0, x)$ né il vecchio suffisso $[y, n)$, mette un valore non negativo in $y-1$ e lascia invariate le molteplicità. Dopo l'arretramento di $y$, quindi, il nuovo suffisso è tutto non negativo. Quando $x = y - 1$ lo scambio riguarda una posizione con sé stessa, e funziona ugualmente.

Ogni ramo diminuisce $y - x$ di 1 mantenendo $0 \le x \le y \le n$, quindi il ciclo termina. All'uscita $x = y$, la zona sconosciuta è vuota, il prefisso contiene soltanto negativi e tutto il resto è non negativo. Nella postcondizione il confine finale $x$ può stare ovunque fra 0 e $n$, estremi compresi. Se tutti i valori sono negativi arriva a $n$, se nessuno lo è resta in 0, e anche l'array vuoto è ammesso.

## 9.6 Per proseguire: bandiera a tre colori

Proviamo ora a separare negativi, zeri e positivi. Alla fine bastano due confini, ma durante il lavoro ne servono tre, perché oltre alle tre zone classificate c'è quella ancora sconosciuta:

$$0 \le x \le z \le y \le n.$$

Manteniamo perm$(a, A)$, con i negativi in $[0, x)$, gli zeri in $[x, z)$ e i positivi in $[y, n)$, mentre la parte sconosciuta è $[z, y)$. Partiamo da $x = z = 0$ e $y = n$, continuiamo finché $z \neq y$ e misuriamo il lavoro rimasto con $y - z$.

L'elemento da esaminare è `a[z]`, e i casi sono tre. Se è negativo, deve andare subito dopo il prefisso dei negativi, cioè in posizione $x$. Scambiamo allora $a[x]$ e $a[z]$ e facciamo avanzare sia $x$ sia $z$. Quando $x < z$, il vecchio $a[x]$ era uno zero, e finisce correttamente in fondo alla zona degli zeri. Quando $x = z$, lo scambio avviene con sé stesso e i due confini avanzano insieme, senza inventare zeri che non esistono.

Se $a[z] = 0$ basta far avanzare $z$. Se infine $a[z] > 0$, arretriamo $y$, scambiamo l'elemento positivo con quello nella nuova posizione $y$ e lasciamo fermo $z$, perché dobbiamo ancora esaminare il valore arrivato da destra, esattamente come nella bandiera a due colori.

```
x := 0; z := 0; y := n;
while z != y do
    if a[z] < 0
        then temp := a[x]; a[x] := a[z]; a[z] := temp;
             x := x + 1; z := z + 1
        else if a[z] = 0
                 then z := z + 1
                 else y := y - 1;
                      temp := a[y]; a[y] := a[z]; a[z] := temp
             fi
    fi
endw
```

Sotto la guardia vale $z < y$, e tutti gli accessi sono validi. In particolare, nel ramo dei positivi il nuovo $y$ è il vecchio $y - 1$, che è almeno $z$. Gli scambi lasciano invariate le molteplicità, e l'analisi dei tre casi mostra che ogni zona mantiene il proprio significato. Ogni ramo diminuisce $y - z$ di 1, quindi il ciclo termina. All'uscita $z = y$, la zona sconosciuta è scomparsa e nell'array troviamo, nell'ordine, i negativi, gli zeri e i positivi. Questo sviluppo completa l'esercizio proposto nelle lezioni.

---

### Verifica 9

**1.** Per `v : array [0,4) of int` con valori $[2, 1, 10, 6]$, esegui `v[v[0]] := v[0]+v[1]`. Indica tutti gli indici controllati e il nuovo array. Spiega perché il comando successivo `v[v[2]] := 0` è definito oppure non lo è.

**2.** Modifica il programma della somma per contare gli elementi negativi di un array. Scrivi un invariante che dia significato al contatore, controlla gli indici e giustifica terminazione e risultato anche per l'array vuoto.

# Soluzioni ragionate

## A.1 Capitolo 1 — Specifiche e stati

**1.1. Prodotto per somme.** Ai controlli della guardia la terna $(A, B, C)$ vale prima $(4, 2, 0)$, poi $(4, 1, 4)$ e infine $(4, 0, 8)$. La variabile $C$ raccoglie la parte del prodotto già calcolata, mentre $B$ conta le aggiunte che mancano.

Il valore iniziale di $B$ era $b = 2$, ma alla fine $B = 0$, e quindi il prodotto $A\,B$, che ora vale 0, non ricorda più quello di partenza. La richiesta corretta parla dei valori iniziali, ed è $C = a\,b = 8$.

**1.2. Il maggiore.** Chiamiamo $x$ e $y$ gli ingressi e $r$ il risultato, e usiamo le lettere di specifica $a$ e $b$ per ricordare i valori iniziali. La precondizione è $x = a \land y = b$, con $a, b \in \mathbb{Z}$. Alla fine vogliamo che gli ingressi siano ancora quelli, cioè di nuovo $x = a \land y = b$, e che

$$r \geq a \land r \geq b \land (r = a \lor r = b).$$

Le due disuguaglianze da sole non bastano, perché ammetterebbero anche numeri troppo grandi. L'ultima condizione chiede che $r$ sia proprio uno dei due ingressi, e insieme le tre condizioni dicono che è il massimo.

La specifica descrive il traguardo ma non dice come raggiungerlo, e lascia aperta la scelta del confronto e degli assegnamenti che produrranno il risultato.

## A.2 Capitolo 2 — Calcolo proposizionale

**2.1. Uscire senza pioggia.** La frase «esco solo se non piove» si scrive $e \Rightarrow \neg p$, perché dice che uscire richiede che non piova. La frase «se non piove, esco» si scrive invece $\neg p \Rightarrow e$, e dice che basterebbe l'assenza di pioggia per farmi uscire.

Per distinguerle prendiamo $p$ ed $e$ entrambe false, cioè non piove e io resto a casa. La prima formula è vera e la seconda falsa, e questo controesempio mostra che la prima non implica la seconda.

**2.2. Semplificare e seguire i segni.** Come nel testo, distribuiamo per avvicinare $P$ alla sua negazione:

$$P \land (\neg P \lor Q) \equiv (P \land \neg P) \lor (P \land Q) \equiv F \lor (P \land Q) \equiv P \land Q.$$

I passi usano nell'ordine la distributività, la contraddizione $P \land \neg P \equiv F$ e la legge di unità.

In $(P \lor \neg Q) \Rightarrow \neg R$, la strada verso $P$ passa per l'antecedente dell'implicazione e poi dentro la disgiunzione, che non inverte. C'è una sola inversione, quindi l'occorrenza è negativa. Per arrivare a $Q$ passiamo per lo stesso antecedente e poi per una negazione, e le due inversioni si annullano, quindi il segno è positivo. Per $R$ passiamo per il conseguente, che non inverte, e poi per una negazione, e il segno è negativo.

Possiamo controllare il risultato eliminando l'implicazione. La formula equivale a $\neg(P \lor \neg Q) \lor \neg R$, cioè a $(\neg P \land Q) \lor \neg R$, dove si vede che $P$ e $R$ compaiono negate e $Q$ no.

## A.3 Capitolo 3 — Individui e formule

**3.1. Sostituire senza catturare.** In $(\forall x.p(x, y)) \land (\exists y.q(x, y))$, dentro $p$ la $x$ è legata e la $y$ è libera. Dentro $q$ accade il contrario, con la $x$ libera e la $y$ legata.

Vogliamo mettere $y$ al posto della sola $x$ libera, che si trova dentro $q$. Se lo facessimo subito, la $y$ inserita verrebbe catturata dall'esistenziale, quindi rinominiamo prima la variabile legata in $z$. Poi sostituiamo e otteniamo

$$(\forall x.p(x,y)) \wedge (\exists z.q(y,z)).$$

La prima parte non cambia, perché non contiene $x$ libere, e le due $y$ rimaste sono entrambe libere, dato che $\exists z$ non le lega.

**3.2. Una proprietà generale e un unico testimone.** Con la funzione di successore $s(n) = n+1$ scriviamo

$$\forall x.(\text{pari}(x) \Rightarrow \text{pari}(s(s(x)))).$$

Qui $s(s(x))$ indica un numero ed è quindi un termine, mentre l'intera scrittura esprime una proprietà ed è una formula chiusa.

Per dire che esattamente un elemento soddisfa $p$ usiamo

$$\exists x.(p(x) \wedge \forall y.(p(y) \Rightarrow y = x)).$$

La parte $p(x)$ garantisce che un testimone esista. L'universale aggiunge che ogni altro elemento con la stessa proprietà coincide con lui, ed è l'uguaglianza a esprimere questa unicità.

## A.4 Capitolo 4 — Semantica e quantificatori

**4.1. Trovare un controesempio.** Neghiamo prima l'universale e poi l'implicazione,

$$\neg(\forall x.(p(x) \Rightarrow \exists y.q(x,y))) \equiv \exists x.(p(x) \wedge \neg(\exists y.q(x,y))),$$

e con un'altra legge di De Morgan portiamo la negazione fino agli atomi:

$$\exists x.(p(x) \wedge \forall y.\neg q(x,y)).$$

Per mostrare falsa la formula iniziale dobbiamo quindi trovare un individuo con la proprietà $p$ per cui la relazione $q$ fallisca con tutti gli individui $y$. Non basta trovare una sola coppia per cui $q$ è falsa.

**4.2. Cambiare il testimone o sceglierne uno solo.** La formula $\forall x.\exists y.R(x,y)$ è vera, perché per $x = a$ possiamo scegliere $y = b$ e per $x = b$ possiamo scegliere $y = a$.

La formula $\exists y.\forall x.R(x,y)$ è invece falsa. Il candidato $y = a$ fallisce con $x = a$, e il candidato $y = b$ fallisce con $x = b$. Ogni elemento ha un compagno, ma nessuno è il compagno di tutti.

Per la generalizzazione lavoriamo sui naturali, con $P(x)$ che significa «$x$ è pari» e una costante $d$ che indica 2. La formula $P(d)$ è vera, ma $\forall x.P(x)$ è falsa, e 1 è un controesempio. Il nome $d$ indicava un oggetto con una proprietà particolare, non un individuo generico.

## A.5 Capitolo 5 — Stati, espressioni e comandi

**5.1. Scambiare con somme e differenze.** Partendo da $(x,y) = (3,7)$, i tre comandi producono nell'ordine $(10,7)$, $(10,3)$ e $(7,3)$.

Per capire il comportamento in generale usiamo gli ingressi generici $A$ e $B$. Dopo il primo comando abbiamo $(A+B, B)$. Il secondo legge il nuovo $x$ e calcola $(A+B) - B = A$, portando a $(A+B, A)$. Il terzo calcola $(A+B) - A = B$, e lo stato finale è $(B, A)$. Abbiamo così dimostrato lo scambio per tutti gli interi matematici. Con interi rappresentati da un numero limitato di cifre dovremmo controllare anche che la somma sia rappresentabile.

**5.2. Diminuire non basta.** Partendo da 4 otteniamo $4, 2, 0$, e dopo due giri la guardia è falsa. Partendo da 5 otteniamo invece $5, 3, 1, -1, -3, \ldots$, senza fine.

Il motivo è che sottrarre 2 non cambia la parità. Nel secondo caso ogni valore resta dispari e non può mai essere zero, quindi la guardia rimane vera per sempre. L'esempio mostra che una variabile può diminuire a ogni giro senza mai raggiungere il valore che farebbe uscire dal ciclo.

## A.6 Capitolo 6 — Triple di Hoare e invarianti

**6.1. Risalire attraverso un assegnamento.** Il valore finale di $y$ sarà $2x$. Sostituendolo nella postcondizione $y > 10$ otteniamo $2x > 10$, cioè $x > 5$.

La moltiplicazione è sempre definita, e poiché $x$ è intero la condizione $x > 5$ equivale a $x \geq 6$. L'ipotesi proposta basta quindi, per l'assioma dell'assegnamento, su tutti gli ingressi ammessi.

**6.2. Una divisione che fallisce sempre.** Prima di `y := z div x` servono la definitezza, cioè $x \neq 0$, e la postcondizione $z > 4$, che l'assegnamento non modifica.

Risalendo attraverso `x := 0` sostituiamo 0 al posto di $x$, e otteniamo

$$0 \neq 0 \wedge z > 4.$$

Questa condizione è falsa in ogni stato, e quindi nessun ingresso rende il programma corretto in senso totale. La precondizione proposta, invece, è soddisfatta dallo stato con $z = 5$ e $x = 3$, e partendo da lì il programma tenta comunque una divisione per zero. La risposta è dunque no.

## A.7 Capitolo 7 — Correttezza totale

**7.1. Un contatore che raggiunge n.** Scegliamo $t = n - i$. Dall'invariante segue $i \leq n$, quindi $t \geq 0$.

Se prima del corpo $n - i = V$, dopo l'incremento vale $n - (i + 1) = V - 1$, che è minore di $V$. Inoltre la guardia $i < n$, con valori interi, dà $i + 1 \leq n$, e l'invariante resta vero.

La guardia e il corpo sono sempre definiti, e il singolo incremento termina, quindi il ciclo termina. All'uscita $i \geq n$ e l'invariante $i \leq n$ danno insieme $i = n$.

**7.2. Aggiungere dopo l'incremento.** Con i due comandi scambiati, il corpo aumenta prima $x$ e poi aggiunge il valore nuovo. L'invariante adatto è

$$I \equiv s = \sum_{j=1}^{x} j \wedge 0 \leq x \leq n.$$

All'inizio la somma è vuota e vale zero. Se prima del corpo $x = k$, il corpo porta $x$ a $k + 1$ e aggiunge proprio $k + 1$ a una somma che arrivava fino a $k$, ristabilendo la stessa proprietà con il nuovo contatore.

La funzione di terminazione resta $n - x$. All'uscita $x = n$, e quindi abbiamo sommato gli interi da 1 a $n$. Per $n = 0$ la somma è vuota e il risultato è zero senza eseguire il corpo.

## A.8 Capitolo 8 — Costruire un ciclo

**8.1. Compensare una modifica.** Dopo `x := x-p`, il termine $x\,y$ ha perso $p\,y$. Possiamo compensare la perdita aumentando $q$ di $y$, perché così $p\,q$ guadagna esattamente $p\,y$. Scegliamo quindi $E = q + y$, e in termini dei valori prima dei due comandi

$$(x - p)y + p(q + y) = x\,y - p\,y + p\,q + p\,y = x\,y + p\,q = n.$$

L'espressione $q + y$ non legge $x$, quindi il primo comando non ne cambia il valore. Quando $p = 0$ l'uguaglianza resterebbe vera con qualunque nuovo valore di $q$, e in particolare anche con quello scelto. Tutte le espressioni sono definite sugli interi.

**8.2. Fibonacci includendo zero.** Mettiamo il risultato in `y`, inizializziamo con `x := 1; i := 0; y := 0` e scegliamo

$$I \equiv x = \text{fib}(i+1) \wedge y = \text{fib}(i) \wedge 0 \le i \le k.$$

Ora entrambi gli indici della successione sono non negativi anche all'inizio, e il corpo resta `temp := x; x := x+y; y := temp; i := i+1`. Dopo gli assegnamenti, indicando con $i$ il vecchio contatore, $x$ contiene $\text{fib}(i+2)$ e $y$ contiene $\text{fib}(i+1)$. Con il nuovo indice $i' = i+1$ ritroviamo esattamente l'invariante.

Sotto la guardia $i \neq k$ i limiti danno $i < k$, quindi l'incremento non supera $k$, e la funzione $k - i$ dimostra la terminazione. All'uscita $i = k$ e $y = \text{fib}(k)$. Per $k = 0$ la guardia è subito falsa e $y = 0$, quindi anche questo caso è compreso.

## A.9 Capitolo 9 — Array

**9.1. Valutare prima di aggiornare.** Per calcolare l'indice leggiamo `v[0]`, che vale 2. Per il valore a destra leggiamo le posizioni 0 e 1 e otteniamo $2 + 1 = 3$. Gli indici usati sono 0, 1 e 2, tutti validi.

Il comando cambia soltanto la posizione 2, e il nuovo array è $[2, 1, 3, 6]$. Il comando successivo legge `v[2]`, che ora vale 3, e scrive in posizione 3, quindi è definito e lascia $[2, 1, 3, 0]$.

Se avessimo eseguito il secondo comando per primo, avrebbe letto 10 in posizione 2 e avrebbe tentato di scrivere fuori dall'array. L'ordine dei comandi può cambiare anche la definitezza degli accessi.

**9.2. Contare i negativi.** Usiamo $c$ per il numero di negativi incontrati fino a quel momento.

```
x := 0; c := 0;
while x != n do
    if a[x] < 0 then c := c + 1 else skip fi;
    x := x + 1
endw
```

L'invariante chiede $0 \le x \le n$ e dice che $c$ è il numero degli indici $i \in [0, x)$ con $a[i] < 0$. All'inizio il prefisso è vuoto e $c = 0$, quindi l'invariante vale.

Sotto la guardia abbiamo $0 \le x < n$, e la lettura di `a[x]` è valida. Se il nuovo elemento è negativo aumentiamo $c$ di uno, altrimenti lo lasciamo com'è. Dopo l'incremento di $x$ il prefisso comprende esattamente una posizione in più, quella appena esaminata, e il conteggio è di nuovo corretto.

La quantità $n - x$ è non negativa e diminuisce di 1, e il corpo termina senza errori, quindi il ciclo termina. All'uscita $x = n$ e $c$ conta i negativi dell'intero array. Se $n = 0$, il risultato è subito $c = 0$ e non viene letta nessuna posizione.